# Agent算法流程

## 目录


1. 项目概述
   - 1.1 项目来源与性质
   - 1.2 技术栈
   - 1.3 核心特性一览
2. 整体架构
   - 2.1 系统分层架构
   - 2.2 核心数据流向
   - 2.3 Task 类型体系
3. 入口与启动流程
   - 3.1 CLI 参数解析
   - 3.2 Bootstrap 初始化序列
   - 3.3 主应用挂载
4. Agent 核心类型系统
   - 4.1 Agent 定义
   - 4.2 Agent 工具调用结果
   - 4.3 Task 状态机
5. Query 引擎 - Agent 决策核心循环
   - 5.1 核心文件位置
   - 5.2 query() 入口函数
   - 5.3 QueryParams 参数结构
   - 5.4 queryLoop 状态机
   - 5.5 压缩边界跨轮次追踪
   - 5.6 API 调用配置快照
   - 5.7 Thinking 块规则
   - 5.8 max_output_tokens 错误恢复
   - 5.9 QueryEngine.submitMessage() 详解
   - 5.10 StreamEvent 事件类型完整列表
6. 工具系统（Tool System）
   - 6.1 工具定义接口
   - 6.2 工具注册表
   - 6.3 工具名称常量
   - 6.4 核心工具列表
7. AgentTool - 子 Agent 启动核心
   - 7.1 工具定义
   - 7.2 Agent 类型注册表
   - 7.3 内置 Agent 类型常量
   - 7.4 验证与错误处理
   - 7.5 异步 Agent 注册流程

- 7.6 权限模式

8. 内置 Agent（Built-in Agents）
    - 8.1 General Purpose Agent
    - 8.2 Explore Agent（一次性）
    - 8.3 Plan Agent（一次性）
    - 8.4 Verification Agent（实验性）
    - 8.5 Claude Code Guide Agent
9. runAgent - 本地 Agent 执行引擎
    - 9.1 完整函数签名
    - 9.2 系统提示词构建
    - 9.3 enhanceSystemPromptWithEnvDetails
    - 9.4 工具过滤逻辑
    - 9.5 核心 query 调用
    - 9.6 进度追踪与 TTFT 指标
    - 9.7 元数据写入
10. LocalAgentTask - 异步 Agent 生命周期管理
    - 10.1 完整状态结构
    - 10.2 异步 Agent 注册
    - 10.3 AbortController 层级
    - 10.4 进度追踪器
    - 10.5 任务通知格式
    - 10.6 任务输出到磁盘
11. RemoteAgent 与 InProcessTeammate
    - 11.1 RemoteAgentTask
    - 11.2 InProcessTeammateTask
12. StreamingToolExecutor - 工具并行编排引擎
    - 12.1 核心设计原则
    - 12.2 核心数据结构
    - 12.3 并发安全性判断
    - 12.4 执行队列算法
    - 12.5 单工具执行流程
    - 12.6 错误传播与兄弟进程中止
    - 12.7 结果聚合
13. 多 Agent 协调模式（Coordinator）
    - 13.1 启用条件
    - 13.2 Coordinator 系统提示词（完整结构）
    - 13.3 Coordinator 工具上下文
    - 13.4 Worker 工具白名单
    - 13.5 内部工具（Worker 不可见）
14. 上下文压缩系统（Compaction）
    - 14.1 压缩策略体系
    - 14.2 精确阈值

- 14.3 自动压缩阈值计算
- 14.4 压缩执行算法
- 14.5 压缩后的墓碑机制
- 14.6 压缩熔断器（Circuit Breaker）
- 14.7 压缩触发的 transcript 刷写

15. 记忆系统（Memory）
    - 15.1 记忆类型
    - 15.2 记忆目录结构
    - 15.3 记忆文件格式
    - 15.4 记忆加载流程
    - 15.5 记忆与系统提示词的关系
    - 15.6 团队记忆（Team Memory）
16. 权限系统（Permissions）
    - 16.1 权限类型
    - 16.2 权限级别
    - 16.3 权限检查流程
    - 16.4 Agent 权限模式
    - 16.5 权限拒绝追踪
17. 状态管理（AppStateStore）
    - 17.1 完整状态结构
    - 17.2 状态更新模式
    - 17.3 Team 上下文状态
18. Slash 命令系统
    - 18.1 命令注册
    - 18.2 主要命令列表
    - 18.3 命令执行流程
19. API 调用层
    - 19.1 Claude API 客户端
    - 19.2 模型配置
    - 19.3 Token 预算管理
20. 关键算法流程详图
    - 20.1 完整 Query Loop 流程
    - 20.2 工具并行/串行决策流程
    - 20.3 多 Agent 协调流程
21. 总结
    - 21.1 Agent 算法核心要点
    - 21.2 关键文件索引
    - 21.3 特色工程设计


---

# 1. 项目概述

## 1.1 项目来源与性质

本项目是通过 npm 包的 source map 文件（`cli.js.map`）逆向还原出的 TypeScript 源代码快照，对应 `@anthropic-ai/claude-code@2.1.88` 版本。源码经过 TypeScript 编译和 bundler 打包后以单个 CLI 文件发布，通过解析 source map 可还原出原始 TypeScript 源码，存储于 `restored-src/src/` 目录（包含 1884 个 `.ts` / `.tsx` 文件）。

这不是一个活跃开发中的仓库，而是一个已发布包的源码快照，但通过分析其结构可以完整理解 Claude Code 的工程实现。

## 1.2 技术栈

| 层级 | 技术选型 |
|---|---|
| 运行时 | Node.js / Bun |
| 语言 | TypeScript + JSX |
| CLI TUI | Ink（React for CLI） |
| AI 接口 | Anthropic Claude API（2023-06-01 版本） |
| 协议 | MCP（Model Context Protocol） |
| 状态管理 | React Context + 自定义 Store |
| 实验特性 | Statsig Feature Gates（A/B 测试框架） |

## 1.3 核心特性一览

- **多模型支持**：Sonnet / Opus / Haiku，支持 fallback 模型降级
- **流式响应**：API 端到端 streaming，减少首 token等待时间（TTFT）
- **40+ 内置工具**：文件系统、Shell、搜索、MCP、代码、LSP 等
- **子 Agent 系统**：支持本地子进程 Agent、远程 Agent、同进程 Teammate
- **多 Agent 协调**：Coordinator 模式下一对多 Worker 协作
- **上下文压缩**：Auto Compact / Reactive Compact / History Snip 多级压缩 -> Reactive Compact是目前没有开放的功能
- **记忆系统**：跨会话持久化记忆（user / feedback / project / reference 四类）
- **权限管理**：细粒度命令级权限控制
- **结构化输出**：支持 `SYNTHETIC_OUTPUT_TOOL_NAME` 工具的结构化结果

> **Note**
>
> 远程agent是指运行在anthropic云端的子agent
>
> - 主agent只能在本地
> - 其spawn的子agent可能在本地可能在云端
>
> | | 远程 | 本地 |
> |---|---|---|
> | 关键词 | 长、独立、需要隔离、耗时 | 快、交互、需要本地状态 |
> | 例子 | 跑测试套件、迁移脚本、批量重构 | 看 git 状态、改代码、发 PR |
>
> 决定条件：
>
> - AgentTool 是本地/远程的唯一分发点
>  - isolation: 'remote' → 调用 teleportToRemote() → 任务发到 CCR 云端执行
>  - 不传 / isolation: 'worktree' → 本地 REPL 进程执行

- 只有两个例外绕过了 AgentTool 直接调 teleportToRemote()：/ultrareview 和 /ultraplan

---

# 2. 整体架构

## 2.1 系统分层架构

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        CLI 入口层 (entrypoints/)                          │
│     cli.tsx → 解析命令行参数 → 分发到不同模式                               │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    │
          ┌──────────────────────────┼──────────────────────────┐
          ▼                          ▼                          ▼

┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│  --dump-prompts  │   │  --agent-worker= │   │     标准交互模式     │
│    (打印提示词)    │   │  (Agent Worker)  │   │                  │
└──────────────────┘   └──────────────────┘   └──────────────────┘
                                │                          │
                                │                          │
                                ▼                          ▼

                     ┌──────────────────┐   ┌──────────────────┐
                     │ Bootstrap 层     │   │  main.tsx (Ink/React)│
                     │ init.ts          │   │   TUI 渲染          │
                     │ setup.ts         │   └──────────────────┘
                     └──────────────────┘              │
                                │                      ▼
                                │
              ┌─────────────────┼──────────────────────────┐
              │                 │                          │
              ▼                 ▼                          ▼

    ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
    │ MCP Servers      │   │  AppStateStore   │   │  QueryEngine     │
    │ (服务连接层)       │   │  (全局状态)       │   │  (决策核心)        │
    └──────────────────┘   └──────────────────┘   └──────────────────┘
                                                          │
                                                          │
              ┌──────────────────────────┼──────────────────────────┐
              │                          │                          │
              ▼                          ▼                          ▼

    ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
    │ StreamingTool    │   │ API Client       │   │ Compaction       │
    │ Executor         │   │ (Claude API)     │   │ Service          │
    │ (工具编排)         │  │                  │  │                  │
    └──────────────────┘   └──────────────────┘   └──────────────────┘
              │
              │
    ┌─────────┼──────────────────────────────────────────┐
    │                                                    │
    ▼                                                    ▼

┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│ 文件系统工具       │  │ Shell 工具        │  │ MCP 协议工具       │
│ Read/Write/Edit  │  │ Bash/PowerShell  │  │ MCP Tool         │
│ Glob/Grep        │  │                  │  │                  │
└──────────────────┘   └──────────────────┘   └──────────────────┘
```

## 2.2 核心数据流向

```
用户输入
    │
    ▼
submitMessage() → query() [AsyncGenerator]
    │
    ├── yield StreamEvent / Message  (流式事件)
    │       ├── text delta          (文本片段)
    │       ├── tool_use block      (工具调用请求)
    │       ├── tool_result         (工具执行结果)
    │       └── message_update      (状态更新)
    │
    └── 返回 Terminal (结束状态)
```

## 2.3 Task 类型体系

文件：`types/Task.ts` 及各 Task 实现文件

```
// Task 类型的完整枚举
type TaskType =
  | 'local_agent'         // 本地子 Agent（AgentTool 启动）
  | 'local_bash'          // 本地 Shell 命令
  | 'remote_agent'        // 远程 CCR 环境中的 Agent
  | 'in_process_teammate' // 同进程内的 Teammate（内存级，无子进程）
  | 'local_workflow'      // 本地工作流脚本
  | 'monitor_mcp'         // MCP 监控任务
  | 'dream'               // 梦境/自主探索模式
```

**Dream / Auto Dream**:

- Dream 是 Claude Code 的自动记忆整合（Memory Consolidation）机制——它在后台静默运行一个子 agent，定期整理和更新记忆文件
- 工作流程
  - Orient — 查看记忆目录结构，读入口索引，了解已有记忆
  - Gather — 从每日日志、矛盾事实、对话记录中收集新信号
  - Consolidate — 将新信息写入/更新记忆文件，合并重复内容
  - Prune — 精简入口索引（memdir.md），删除过时记忆，修复矛盾
- 触发同条件

| 门控 | 条件 | 说明 |
|---|---|---|
| **特性开关** | GrowthBook tengu_onyx_plover 或用户设置 autoDreamEnabled | 总开关 |
| **时间门** | 距上次整合 ≥ minHours（默认 24h） | 避免过于频繁 |
| **扫描节流** | 上次扫描 ≥ 10 分钟 | 防止时间门通过但会话门未通过时重复触发 |
| **会话门** | 新增会话数 ≥ minSessions（默认 5 个） | 确保有足够素材可整合 |
| **锁文件** | 无其他进程正在整合（PID 文件 + 过期 1h 自动回收） | 防并发 |

每种 Task 类型对应一个独立的 Task 组件文件，统一通过 `AppStateStore.tasks` 字典管理生命周期。

---

# 3. 入口与启动流程

## 3.1 CLI 参数解析

文件：`entrypoints/cli.tsx`

```
// 入口分发逻辑
async function main() {
  const parsed = parseArgs(process.argv);

  // 快速路径：版本号
  if (parsed.version || parsed.v) return printVersion();

  // 调试/工具路径：打印渲染后的系统提示词
  if (parsed.dumpPrompts) {
    return createDumpPromptsFetch(parsed.dumpPrompts)();
  }

  // Agent Worker 模式：子 Agent 的执行入口
  // 启动参数：--agent-worker=<taskId> --task-script=<path>
  if (parsed.agentWorker) {
    return runAgentWorker({
      taskId: parsed.agentWorker,
      taskScript: parsed.taskScript,
    });
  }

  // 守护进程工作模式
  if (parsed.daemonWorker) {
    return daemonWorkerLoop();
  }

  // MCP 服务器模式
  if (parsed.mcpServerMode) {
    return runMCPServer(parsed);
  }

  // 标准交互模式
  return runClaudeCodeCLI(parsed);
}
```

## 3.2 Bootstrap 初始化序列

文件：`bootstrap/init.ts`

```
init()
  │
  ├── 检测 CLI 入口点（sdk-ts / sdk-py / sdk-cli / null）
  │      └── 影响内置 Agent 的加载策略（SDK 模式可禁用内置 Agent）
  │
  ├── 加载 Claude Code 数据目录 ~/.claude/
  │      ├── settings.json
  │      ├── memory/  （记忆文件）
```

```
│      ├── commands/ (自定义命令)
│      └── skills/   (技能定义)
│
├── 检测运行模式
│      ├── CLAUDE_CODE_REMOTE=1 → RemoteAgentTask 启动
│      ├── CLAUDE_CODE_COORDINATOR_MODE=1 → Coordinator 启动
│      └── 默认 → 主交互 Session
│
├── 初始化 MCP 服务器
│      ├── 检测 MCP_SERVERS 环境变量
│      ├── 检测 ~/.claude/mcp_servers.json
│      └── 启动 MCP 连接（最多等待 30s）
│
└── 初始化 Statsig GrowthBook（A/B 测试）
       └ 获取实验特性开关：COORDINATOR_MODE / REACTIVE_COMPACT 等
```

> **Note**
>
> Bootstrap 层是 Claude Code 启动时从 Anthropic 服务器拉取**启动配置**并持久化到本地的机制，确保应用在启动后能立即获得最新的服务端配置

### 3.3 主应用挂载

文件：`main.tsx`（编译后的大文件，约 803KB）

使用 **Ink**（React for CLI）渲染 TUI：

```
<AppStateStore.Provider initialState={appState}>
  <ThemeProvider>
    <ScreenRouter>
      {/* 主交互屏幕 */}
      <MainScreen />
      {/* 配置编辑屏幕 */}
      <ConfigScreen />
      {/* 帮助屏幕 */}
      <HelpScreen />
      {/* Agent Team 视图 */}
      <TeamScreen />
    </ScreenRouter>
  </ThemeProvider>
</AppStateStore.Provider>
```

---

## 4. Agent 核心类型系统

### 4.1 Agent 定义

文件：`tools/AgentTool/loadAgentsDir.ts`

```
// Agent 的核心定义
interface AgentDefinition {
  agentType: string;            // 唯一标识符，如 "general-purpose"
  whenToUse?: string;            // Agent 选择提示（供 LLM 理解何时使用）
```

```
  tools: string[] | ['*'];       // 可用工具列表，'*' 表示全部
  source: 'built-in' | string;  // 来源：内置 / 用户自定义目录
  baseDir: string;               // 定义文件所在目录
  model?: string;                // 可选：指定模型
  maxTurns?: number;             // 最大对话轮次
  permissionMode?: PermissionMode; // 权限模式

  // 工厂函数：动态构建系统提示词
  getSystemPrompt(
    context: { toolUseContext: Pick<ToolUseContext, 'options'> }
  ): string;
}
```

需要动态加载的上下文：

**动态加载的 12 类上下文**

| # | 上下文类型 | 来源 | 缓存策略 |
|---|---|---|---|
| 1 | **技能/工具使用策略** | skillToolCommands + enabledTools | 缓存 |
| 2 | **记忆内容** | .claude/memdir/ 目录 | 缓存 |
| 3 | **环境信息** | 实时读取：cwd、git状态、平台、shell、OS、模型ID | 缓存 |
| 4 | **语言偏好** | settings.json → language | 缓存 |
| 5 | **输出风格配置** | output style 插件 | 缓存 |
| 6 | **MCP 服务器指令** | 已连接 MCP server 的 instructions | **每轮重算**（MCP可动态连接/断开） |
| 7 | **Scratchpad 目录** | settings.json → permissions.scratchpad | 缓存 |
| 8 | **模型知识截止日期** | 根据模型 ID 映射 | 缓存 |
| 9 | **Token 预算指令** | GrowthBook feature flag | 缓存 |
| 10 | **Proactive/Brief 指南** | KAIROS 特性开关 | 缓存 |
| 11 | **会话钩子说明** | 用户配置的 hooks | 缓存 |
| 12 | **Agent 特定 MCP servers** | frontmatter 中定义的 MCP 配置 | 运行时一次性初始化 |

关键设计

- 静态/动态分离：SYSTEM_PROMPT_DYNAMIC_BOUNDARY 标记之前的静态部分可用于跨组织的提示词缓存（节省约 90% token 成本）
- MCP 特殊处理：MCP instructions 每轮重算，因为 MCP 服务器可能在对话中途连接/断开
- toolUseContext 的作用：传递 options（含 mainLoopModel、mcpClients、agentDefinitions 等），使 getSystemPrompt 能根据运行时状态做决策

## 4.2 Agent 工具调用结果

文件：`tools/AgentTool/AgentTool.tsx`

```
// AgentTool 同步调用的返回结果
type AgentToolResult =
  | { status: 'success'; result: string }
  | { status: 'async_launched'; agentId: string; outputFile: string }
  | { status: 'error'; error: string };

// 异步 Agent 的最终输出
interface AsyncAgentResult {
  status: 'completed' | 'failed' | 'killed';
  summary: string;       // 任务摘要（用于通知父 Agent）
  result?: string;        // 完整结果
```

```
  usage?: {
    total_tokens: number;
    tool_uses: number;
    duration_ms: number;
  };
}
```

### 4.3 Task 状态机

```
// Task 的生命周期状态
type TaskStatus =
  | 'pending'    // 已注册，等待启动
  | 'running'    // 执行中
  | 'completed'  // 正常完成
  | 'failed'     // 执行失败
  | 'killed'     // 被手动停止
  | 'paused';    // 暂停（等待用户确认）

// LocalAgentTaskState 完整结构
interface LocalAgentTaskState extends TaskStateBase {
  type: 'local_agent';
  agentId: string;                    // Agent ID
  agentType: string;                   // Agent 类型
  prompt: string;                      // 初始 prompt
  selectedAgent?: AgentDefinition;    // Agent 定义对象
  model?: string;                      // 指定模型
  abortController?: AbortController;   // 中止控制器
  error?: string;                     // 错误信息
  result?: AgentToolResult;           // 最终结果
  progress?: AgentProgress;            // 实时进度
  retrieved: boolean;                  // 输出是否已被读取
  messages?: Message[];                // Agent 消息历史
  isBackgrounded: boolean;            // 是否后台运行
  pendingMessages: string[];           // 待处理消息队列
  retain: boolean;                     // 是否保留（不受 evict 影响）
  diskLoaded: boolean;                 // 是否从磁盘加载过
  evictAfter?: number;                 // N 次访问后驱逐
  lastReportedToolCount: number;       // 上次报告的工具调用数
  lastReportedTokenCount: number;      // 上次报告的 Token 数
}
```

---

## 5. Query 引擎 - Agent 决策核心循环

### 5.1 核心文件位置

| 文件 | 行数 | 作用 |
|---|---|---|
| `query.ts` | ~800+ | `query()` 函数定义 + `queryLoop()` 核心循环 |
| `QueryEngine.ts` | ~1300+ | `QueryEngine` 类的完整实现 |
| `query/config.ts` | - | `buildQueryConfig()` 查询配置构建 |
| `query/deps.ts` | - | `productionDeps` 依赖注入 |

| 文件 | 行数 | 作用 |
| --- | --- | --- |
| `query/stopHooks.ts` | - | 停止钩子处理 |
| `query/tokenBudget.ts` | - | Token 预算管理 |

## 5.2 query() 入口函数

文件：`query.ts`（第 219-239 行）

```
export async function* query(
  params: QueryParams,
): AsyncGenerator<
  | StreamEvent        // 流式事件（text / tool_use / tool_result 等）
  | RequestStartEvent  // 请求开始事件
  | Message            // 消息对象
  | TombstoneMessage   // 墓碑消息（已压缩历史的占位符）
  | ToolUseSummaryMessage, // 工具调用摘要
  Terminal            // 返回值类型
> {
  const consumedCommandUuids: string[] = [];

  // 调用 queryLoop，执行实际的流式循环
  const terminal = yield* queryLoop(params, consumedCommandUuids);

  // 通知所有消耗的命令完成
  for (const uuid of consumedCommandUuids) {
    notifyCommandLifecycle(uuid, 'completed');
  }

  return terminal;
}
```

**关键设计**：使用 TypeScript Generator + `yield*`（委托生成器）实现异步流式处理。`yield*` 将子生成器的值直接向上传递，直到子生成器返回 `Terminal` 对象。

## 5.3 QueryParams 参数结构

```
export type QueryParams = {
  messages: Message[];              // 对话历史消息
  systemPrompt: SystemPrompt;       // 系统提示词（已渲染）
  userContext: { [k: string]: string };    // 用户上下文（会话相关）
  systemContext: { [k: string]: string };  // 系统上下文（环境相关）
  canUseTool: CanUseToolFn;          // 权限检查函数
  toolUseContext: ToolUseContext;    // 工具执行上下文
  fallbackModel?: string;           // API 调用失败时的降级模型
  querySource: QuerySource;         // 查询来源（main / agent / sdk）
  maxOutputTokensOverride?: number;  // 最大输出 Token 数覆盖
  maxTurns?: number;                // 最大对话轮次
  skipCacheWrite?: boolean;         // 跳过缓存写入
  taskBudget?: { total: number };   // API task_budget（beta）
  deps?: QueryDeps;                  // 依赖注入（可测试性）
};
```

## 5.4 queryLoop 状态机

文件：`query.ts`（第 241 行起）

```
type State = {
  messages: Message[];                          // 对话历史（可变）
  toolUseContext: ToolUseContext;               // 工具执行上下文
  autoCompactTracking: AutoCompactTrackingState | undefined;
  maxOutputTokensRecoveryCount: number;          // max_output_tokens 恢复计数
  hasAttemptedReactiveCompact: boolean;          // 是否已尝试反应式压缩
  maxOutputTokensOverride: number | undefined;  // 最大输出覆盖
  pendingToolUseSummary: Promise<ToolUseSummaryMessage | null> | undefined;
  stopHookActive: boolean | undefined;          // 停止钩子是否激活
  turnCount: number;                            // 当前轮次（从 1 开始）
  transition: Continue | undefined;             // 上次迭代为何继续
};
```

**初始状态设置**（第 268-279 行）：

```
let state: State = {
  messages: params.messages,
  toolUseContext: params.toolUseContext,
  maxOutputTokensOverride: params.maxOutputTokensOverride,
  autoCompactTracking: undefined,
  stopHookActive: undefined,
  maxOutputTokensRecoveryCount: 0,
  hasAttemptedReactiveCompact: false,
  turnCount: 1,
  pendingToolUseSummary: undefined,
  transition: undefined,
};
```

## 5.5 压缩边界跨轮次追踪

```
// task_budget.remaining: 跨压缩边界追踪剩余预算
// 压缩前: API server 自己维护从 {total} 开始的倒计时
// 压缩后: server 只看到摘要，remaining 告知它压缩前的实际消耗
let taskBudgetRemaining: number | undefined = undefined;
```

## 5.6 API 调用配置快照

```
// 在 query 入口处快照所有不可变的配置
// 避免每轮迭代都重新读取环境变量
const config = buildQueryConfig();

// buildQueryConfig 快照内容:
// - 模型配置（主循环模型 / fallback 模型）
// - 上下文窗口大小
// - API 限额配置
// - 工具列表
// 注意: feature() 特性开关故意不在快照中（实时读取）
```

## 5.7 Thinking 块规则

文件：`query.ts`（第 151-163 行）

```
/**
 * Thinking 块的规则 ("wizard rules"):
 * 1. 包含 thinking / redacted_thinking 块的消息必须 max_thinking_length > 0
 * 2. Thinking 块不能是消息块中的最后一个块
 * 3. Thinking 块必须在整个 assistant 轨迹中被保留
 *     (单个 turn, 或该 turn 包含 tool_use + 其 tool_result + 后续 assistant message)
 */
```

## 5.8 max_output_tokens 错误恢复

```
const MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3;

// 检测: 是否是 max_output_tokens 错误 (需要延迟报告)
function isWithheldMaxOutputTokens(
  msg: Message | StreamEvent | undefined,
): msg is AssistantMessage {
  return msg?.type === 'assistant' && msg.apiError === 'max_output_tokens';
}

// 恢复逻辑:
// 如果收到 max_output_tokens 错误 → 尝试恢复 → 最多重试 3 次
// 重试策略: 增大 maxOutputTokensOverride (使用 ESCALATED_MAX_TOKENS)
```

## 5.9 QueryEngine.submitMessage() 详解

文件：`QueryEngine.ts`

```
async *submitMessage(
  prompt: string | ContentBlockParam[],
  options?: { uuid?: string; isMeta?: boolean }
): AsyncGenerator<SDKMessage, void, unknown>
```

**流程:**

```
1. 追加用户消息到 mutableMessages[]
   └ 若 prompt 是 string → 包装为 text ContentBlock (即对原始内容进行结构化解析)
   └ 若 prompt 是 ContentBlockParam[] → 直接追加

2. 构建系统提示词
   └ fetchSystemPromptParts() 获取三部分:
       - defaultSystemPrompt (静态部分)
       - userContext (用户相关)
       - systemContext (系统相关)
   └ 合并 coordinatorUserContext (若启用协调模式)
   └ 注入 loadMemoryPrompt() (若自定义 MEM.AUTO_MEM_PATH)

3. 包装 canUseTool (权限检查包装器)
   └ 追踪 permissionDenials[] (SDK 返回给 SDK 调用者)
```

```
4. 调用 query()
   └─ 传入所有构建好的参数

5. 处理 StreamEvent / Message
   └─ 过滤 API error retry 事件（SDKStatus）
   └─ 追加 TTFT 指标到 toolUseContext
   └─ 更新 messageIndexMap
   └─ 追踪 token 使用量到 totalUsage

6. 处理完成/错误结果
   └─ CompactBoundary → 刷入 memory transcript
   └─ error_max_turns → 达到最大轮次
   └─ error_max_budget_usd → 超出 USD 预算
   └─ error_during_execution → 执行异常
```

### 5.10 StreamEvent 事件类型完整列表

```typescript
// text 片段
type TextEvent = { type: 'text'; content: string; isOpen: boolean };

// 工具调用
type ToolUseEvent = { type: 'tool_use'; tool: ToolUseBlock };

// 工具结果（用于同步工具）
type ToolResultEvent = { type: 'tool_result'; result: ToolResultBlockParam };

// 错误事件
type ErrorEvent = { type: 'error'; error: string; code?: string };

// 完成事件
type DoneEvent = { type: 'done' };

// 悬停预览（工具调用前的预览）
type PendingToolUseEvent = { type: 'pending_tool_use'; tool: ToolUseBlock };

// 进度更新
type ProgressEvent = { type: 'progress'; progress: number };

// 推测更新（prefill / 实时补全）
type SpeculationEvent = { type: 'speculation_update'; spec: Speculation };

// API 指标
type ApiMetricsEvent = {
  type: 'message_start';
  message: AssistantMessage;
  ttftMs: number;  // Time to First Token（首 token 延迟）
};
```

---

## 6. 工具系统（Tool System）

> **Note**
>
> 详见 “工具解析” 相关文档

## 6.1 工具定义接口

文件：`Tool.ts`

```
interface Tool {
  name: string;                          // 唯一名称
  description: string;                   // 描述（供 LLM 理解用途）
  matchesCapability?: Capability;        // 匹配的能力标记
  inputSchema: ZodSchema;               // 输入参数 Zod Schema

  // 权限相关
  requiresPermissions?: Permission[];
  allowedWhenPaused?: boolean;           // 暂停时是否可执行

  // 执行入口
  invoke(input: unknown, context: ToolUseContext): Promise<ToolResult>;
  stream?(
    input: unknown,
    context: ToolUseContext
  ): AsyncGenerator<Partial<ToolResult>>; // 流式执行

  // 可选：并发安全性判断
  isConcurrencySafe?(
    input: unknown
  ): boolean;
}

interface ToolUseContext {
  abortController: AbortController;
  toolUseId: string;
  options: {
    cwd: string;
    env: Record<string, string>;
    sessionId: string;
    // ... 更多上下文
  };
  pushApiMetricsEntry?: (ttftMs: number) => void;
}
```

## 6.2 工具注册表

```
class ToolRegistry {
  // 核心数据结构
  private tools: Map<string, Tool> = new Map();

  // 注册
  register(tool: Tool): void;
```

```
  // 按名称查找
  get(name: string): Tool | undefined;

  // 全部获取
  getAll(): Tool[];

  // 按 Agent 过滤（权限）
  getToolsForAgent(agentId: string, permissions: Permission[]): Tool[];

  // 按能力过滤
  getToolsByCapability(capability: Capability): Tool[];
}

// 查找工具（带辅助信息）
function findToolByName(tools: Tools, name: string): Tool | undefined;
```

## 6.3 工具名称常量

文件：`constants/tools.ts`

```
export const FILE_READ_TOOL_NAME = 'Read';
export const FILE_WRITE_TOOL_NAME = 'Write';
export const FILE_EDIT_TOOL_NAME = 'Edit';
export const BASH_TOOL_NAME = 'Bash';
export const GREP_TOOL_NAME = 'Grep';
export const GLOB_TOOL_NAME = 'Glob';
export const TASK_CREATE_TOOL_NAME = 'TaskCreate';
export const TODO_WRITE_TOOL_NAME = 'TodoWrite';
export const AGENT_TOOL_NAME = 'Agent';
export const LEGACY_AGENT_TOOL_NAME = 'Task';
export const SKILL_TOOL_NAME = 'Skill';
export const SEND_MESSAGE_TOOL_NAME = 'SendMessage';
export const TASK_STOP_TOOL_NAME = 'TaskStop';
export const TEAM_CREATE_TOOL_NAME = 'TeamCreate';
export const TEAM_DELETE_TOOL_NAME = 'TeamDelete';
exPS：==详见 工具解析 那个文件==port const SYNTHETIC_OUTPUT_TOOL_NAME = 'SyntheticOutput';
```

## 6.4 核心工具列表

| 工具 | 文件位置 | 功能描述 |
|---|---|---|
| `Read` | `tools/FileReadTool/` | 读取文件（支持行范围、图像） |
| `Write` | `tools/FileWriteTool/` | 写入/覆盖文件 |
| `Edit` | `tools/FileEditTool/` | 精确字符串替换编辑 |
| `Glob` | `tools/GlobTool/` | 按模式匹配文件路径 |
| `Grep` | `tools/GrepTool/` | 内容搜索（支持正则） |
| `Bash` | `tools/BashTool/` | 执行 Shell 命令 |
| `Agent` | `tools/AgentTool/` | 启动子 Agent（核心） |
| `SendMessage` | `tools/SendMessageTool/` | 向运行中的 Agent 发消息 |
| `TaskStop` | `tools/TaskStopTool/` | 停止 Agent 任务 |
| `Skill` | `tools/SkillTool/` | 调用 Skill |

| 工具 | 文件位置 | 功能描述 |
| --- | --- | --- |
| `MCP` | `tools/McpTool/` | 调用 MCP 工具 |
| `WebSearch` | `tools/WebSearchTool/` | 互联网搜索 |
| `WebFetch` | `tools/WebFetchTool/` | 抓取网页 |
| `TodoWrite` | `tools/TodoWriteTool/` | 任务清单管理 |
| `AskUserQuestion` | `tools/AskUserQuestionTool/` | 向用户提问 |
| `TaskCreate` | `tools/TaskCreateTool/` | 创建 Task |
| `TaskOutput` | `tools/TaskOutputTool/` | 读取 Agent 输出 |
| `TeamCreate` | `tools/TeamCreateTool/` | 创建 Agent Team |
| `Compact` | `commands/compact/` | 手动触发压缩 |
| `Config` | `tools/ConfigTool/` | 配置管理 |
| `LSP` | `tools/LSPTool/` | Language Server Protocol |
| `NotebookEdit` | `tools/NotebookEditTool/` | Jupyter 编辑 |
| `Sleep` | `tools/SleepTool/` | 延迟执行 |

---

# 7. AgentTool - 子 Agent 启动核心

> **Note**
>
> 详见 sub-agent 的相关文档

## 7.1 工具定义

文件：`tools/AgentTool/AgentTool.tsx`

AgentTool 是整个子 Agent 系统的入口工具，定义如下：

```
// 输入 Schema（Zod）
const baseInputSchema = z.object({
  description: z.string().describe('简短描述（3-5 词）'),
  prompt: z.string().describe('Agent 的任务描述'),
  subagent_type: z.string().optional(),     // Agent 类型（默认 general-purpose）
  model: z.enum(['sonnet', 'opus', 'haiku']).optional(),  // 指定模型
  run_in_background: z.boolean().optional() // 后台运行
});

// 完整 Schema（Team 模式增加字段）
const fullInputSchema = baseInputSchema.merge(z.object({
  name: z.string().optional(),              // Agent 名称（用于 SendMessage 路由）
  team_name: z.string().optional(),        // Team 名称
  mode: permissionModeSchema().optional(),  // 权限模式
  isolation: z.enum(['worktree', 'remote']).optional(), // 隔离模式
  cwd: z.string().optional()                // 工作目录
}));
```

## 7.2 Agent 类型注册表

文件：`tools/AgentTool/builtInAgents.ts`

```
export function getBuiltInAgents(): AgentDefinition[] {
  // SDK 模式可禁用所有内置 Agent
  if (isNonInteractiveSession &&
      CLAUDE_AGENT_SDK_DISABLE_BUILTIN_AGENTS) {
    return [];
  }

  // Coordinator 模式：返回协调器 Agent
  if (feature('COORDINATOR_MODE') &&
      process.env.CLAUDE_CODE_COORDINATOR_MODE) {
    return getCoordinatorAgents();
  }

  const agents = [
    GENERAL_PURPOSE_AGENT,
    STATUSLINE_SETUP_AGENT,
  ];

  // Explore / Plan Agent（A/B 测试控制）
  if (areExplorePlanAgentsEnabled()) {
    agents.push(EXPLORE_AGENT, PLAN_AGENT);
  }

  // Claude Code Guide（非 SDK 入口点才加载）
  if (isNonSdkEntrypoint) {
    agents.push(CLAUDE_CODE_GUIDE_AGENT);
  }

  // Verification Agent（GrowthBook 实验）
  if (feature('VERIFICATION_AGENT') &&
      getFeatureValue('tengu_hive_evidence', false)) {
    agents.push(VERIFICATION_AGENT);
  }

  return agents;
}
```

## 7.3 内置 Agent 类型常量

文件：`tools/AgentTool/constants.ts`

```
// Agent 工具名称
export const AGENT_TOOL_NAME = 'Agent';
export const LEGACY_AGENT_TOOL_NAME = 'Task';  // 向后兼容

// 一次性 Agent（父 Agent 不会继续与其交互）
// 节省 token：~135 chars × 34M runs/week
export const ONE_SHOT_BUILTIN_AGENT_TYPES: ReadonlySet<string> = new Set([
  'Explore',   // 一次性探索
  'Plan',      // 一次性规划
]);
```

```
// 验证 Agent
export const VERIFICATION_AGENT_TYPE = 'verification';
```

## 7.4 验证与错误处理

文件：`AgentTool.tsx`（第 262-409 行）

```
// 错误 1: Agent Teams 未启用
if (team_name && !isAgentSwarmsEnabled()) {
  throw new Error('Agent Teams is not yet available on your plan.');
}

// 错误 2: Teammate 不能 spawn 其他 Teammate
if (isTeammate() && teamName && name) {
  throw new Error('Teammates cannot spawn other teammates...');
}

// 错误 3: In-process Teammate 不能 spawn 后台 Agent
if (isInProcessTeammate() && teamName && run_in_background === true) {
  throw new Error('In-process teammates cannot spawn background agents...');
}

// 错误 4: Agent 类型未找到
throw new Error(`Agent type '${effectiveType}' not found. ` +
  `Available agents: ${agents.map(a => a.agentType).join(', ')}`);

// 错误 5: Agent 被权限规则拒绝
throw new Error(`Agent type '${effectiveType}' has been denied by permission rule...`);

// MCP 服务器依赖检查（第 369-409 行）
// 最多等待 30 秒让 MCP 服务器就绪（POLL_INTERVAL_MS = 500ms）
if (!hasRequiredMcpServers(selectedAgent, serversWithTools)) {
  throw new Error(`Agent '${selectedAgent.agentType}' requires MCP servers...`);
}
```

## 7.5 异步 Agent 注册流程

文件：`AgentTool.tsx`（第 686-764 行）

```
// shouldRunAsync 判断
// 规则: run_in_background=true 或 Team 模式 → 异步

if (shouldRunAsync) {
  // 1. 注册异步 Agent，获得 taskId
  const agentBackgroundTask = registerAsyncAgent({
    agentId: asyncAgentId,
    description,
    prompt,
    selectedAgent,
    setAppState: rootSetAppState,
    toolUseId: toolUseContext.toolUseId,
  });
```

```
  // 2. 注册名称路由（用于 SendMessage）
  if (name) {
    rootSetAppState(prev => {
      const next = new Map(prev.agentNameRegistry);
      next.set(name, asAgentId(asyncAgentId));
      return { ...prev, agentNameRegistry: next };
    });
  }

  // 3. 返回异步启动结果
  return {
    data: {
      status: 'async_launched',
      agentId: agentBackgroundTask.agentId,
      description,
      prompt,
      outputFile: getTaskOutputPath(agentBackgroundTask.agentId),
      canReadOutputFile,
    }
  };
} else {
  // 同步模式：在当前查询中直接运行 Agent
  return { data: { status: 'sync', result: await runAgentSync(...) } };
}
```

### 7.6 权限模式

```
type PermissionMode =
  | 'acceptEdits'    // 接受编辑（默认）
  | 'restricted'    // 受限模式
  | 'browseOnly'     // 仅浏览
  | 'modes.loco';   // 自定义模式

// 权限模式的自动授权策略
// - async Agent with prompts allowed: canShowPermissionPrompts = true
// - bubble mode: canShowPermissionPrompts = false
// - sync Agent: canShowPermissionPrompts = true（默认）
```

---

## 8. 内置 Agent（Built-in Agents）

> **Note**
>
> - AgentTool章节关注的是SubAgent的spawn方式
> - 内置Agent章节关注的是这些内置的SubAgent具体怎么配置的

### 8.1 General Purpose Agent

文件：`tools/AgentTool/built-in/generalPurposeAgent.ts`

```
const SHARED_PREFIX = `You are an agent for Claude Code,
Anthropic's official CLI for Claude. Given the user's message,
```

```
you should use the tools available to complete the task.
Complete the task fully—don't gold-plate, but don't leave it half-done.`;

const SHARED_GUIDELINES = `Your strengths:
- Searching for code, configurations, and patterns across large codebases
- Analyzing multiple files to understand system architecture
- Investigating complex questions that require exploring many files
- Performing multi-step research tasks

Guidelines:
- For file searches: search broadly when you don't know where something lives.
- For analysis: Start broad and narrow down.
- Be thorough: Check multiple locations, consider different naming conventions.
- NEVER create files unless absolutely necessary. Prefer editing existing files.
- NEVER proactively create documentation files (*.md) unless explicitly requested.`;

export const GENERAL_PURPOSE_AGENT: BuiltInAgentDefinition = {
  agentType: 'general-purpose',
  whenToUse:
    'General-purpose agent for researching complex questions, ' +
    'searching for code, and executing multi-step tasks. When you are ' +
    'searching for a keyword or file and are not confident that you will ' +
    'find the right match in the first few tries use this agent.',
  tools: ['*'],       // 所有工具
  source: 'built-in',
  baseDir: 'built-in',
  model: undefined,   // 使用 getDefaultSubagentModel() 默认值
  getSystemPrompt: getGeneralPurposeSystemPrompt,
};
```

## 8.2 Explore Agent（一次性）

```
export const EXPLORE_AGENT = {
  agentType: 'Explore',
  whenToUse: 'Use when searching for a keyword or file...',
  tools: ['Glob', 'Grep', 'Read', 'Bash'],
  source: 'built-in',
  getSystemPrompt: () => `You are a codebase exploration agent...`,
};
// 特点: ONE_SHOT_BUILTIN_AGENT_TYPES 成员，父 Agent 不会继续交互（即“一次性”）
```

## 8.3 Plan Agent（一次性）

```
export const PLAN_AGENT = {
  agentType: 'Plan',
  // 用于规划模式
  tools: ['*'],
  source: 'built-in',
  getSystemPrompt: () => `You are a planning agent...`,
};
// 特点: ONE_SHOT_BUILTIN_AGENT_TYPES 成员
```

## 8.4 Verification Agent（实验性）

```
export const VERIFICATION_AGENT = {
  agentType: 'verification',
  tools: ['Bash', 'Read', 'Grep'],
  source: 'built-in',
  getSystemPrompt: () => `You are a code verification agent...`,
};
// 特点: GrowthBook 实验 'tengu_hive_evidence' 控制开关
```

主agent中关于调用这个verification agent的系统提示词（暂未开启）

```
  getFeatureValue_CACHED_MAY_BE_STALE('tengu_hive_evidence', false)
    ? `### **The Contract**

    When **non-trivial implementation** happens on your turn, independent adversarial
verification **must** happen before you report completion — regardless of who did the
implementing (you directly, a fork you spawned, or a subagent).

    **You are the one reporting to the user; you own the gate.**

    - **Non-trivial means:** 3+ file edits, backend/API changes, or infrastructure changes.

    - **Action:** Spawn the `${AGENT_TOOL_NAME}` tool with
`subagent_type="${VERIFICATION_AGENT_TYPE}"`.

    - **Requirements:** * Your own checks, caveats, and a fork's self-checks do **NOT**
substitute.

        - Only the verifier assigns a verdict; you cannot self-assign `PARTIAL`.

        - Pass the original user request, all files changed (by anyone), the approach, and
the plan file path if applicable.

        - Flag concerns if you have them, but do **NOT** share test results or claim things
work.


    **Handling Results:**

    - **On FAIL:** Fix, resume the verifier with its findings plus your fix, and repeat
until `PASS`.

    - **On PASS:** Spot-check it — re-run 2-3 commands from its report. Confirm every `PASS`
has a Command run block with output that matches your re-run. If any `PASS` lacks a command
block or diverges, resume the verifier with the specifics.

    - **On PARTIAL:** (From the verifier) Report what passed and what could not be
verified.`

    : null,
```

### 8.5 Claude Code Guide Agent

```
export const CLAUDE_CODE_GUIDE_AGENT = {
  agentType: 'claude-code-guide',
  tools: ['WebSearch', 'WebFetch'],
  source: 'built-in',
  getSystemPrompt: () => `You are a guide for Claude Code users...`,
};
// 特点：非 SDK 入口点才加载（isNonSdkEntrypoint 判断）
```

---

## 9. runAgent - 本地 Agent 执行引擎

### 9.1 完整函数签名

文件：`tools/AgentTool/runAgent.ts`（第 248-329 行）

```
export async function* runAgent({
  agentDefinition,          // Agent 定义
  promptMessages,           // 初始消息（可选）
  toolUseContext,            // 工具执行上下文
  canUseTool,               // 权限检查函数
  isAsync,                  // 是否异步模式
  canShowPermissionPrompts, // 是否显示权限对话框
  forkContextMessages,       // 分叉上下文消息
  querySource,              // 查询来源
  override,                 // 覆盖参数
  model,                    // 指定模型
  maxTurns,                 // 最大轮次
  preserveToolUseResults,   // 保留工具结果
  availableTools,           // 可用工具列表
  allowedTools,             // 允许的工具列表
  onCacheSafeParams,        // 缓存安全参数回调
  contentReplacementState,  // 内容替换状态
  useExactTools,            // 使用精确工具列表
  worktreePath,             // Git worktree 路径
  description,              // 任务描述
  transcriptSubdir,         // Transcript 子目录
  onQueryProgress,          // 进度回调
}): AsyncGenerator<Message, void>
```

### 9.2 系统提示词构建

文件：`runAgent.ts`（第 906-932 行）

```
async function getAgentSystemPrompt(
  agentDefinition: AgentDefinition,
  toolUseContext: Pick<ToolUseContext, 'options'>,
  resolvedAgentModel: string,
  additionalWorkingDirectories: string[],
  resolvedTools: readonly Tool[],
): Promise<string[]> {
  const enabledToolNames = new Set(resolvedTools.map(t => t.name));
```

```
  try {
    // 优先使用 Agent 定义自己的 getSystemPrompt
    const agentPrompt = agentDefinition.getSystemPrompt({ toolUseContext });
    const prompts = [agentPrompt];

    return await enhanceSystemPromptWithEnvDetails(
      prompts,
      resolvedAgentModel,
      additionalWorkingDirectories,
      enabledToolNames,
    );
  } catch (_error) {
    // 降级：使用默认 Agent 提示词
    return enhanceSystemPromptWithEnvDetails(
      [DEFAULT_AGENT_PROMPT],
      resolvedAgentModel,
      additionalWorkingDirectories,
      enabledToolNames,
    );
  }
}
```

## 9.3 enhanceSystemPromptWithEnvDetails

```
async function enhanceSystemPromptWithEnvDetails(
  existingSystemPrompt: string[],
  model: string,
  additionalWorkingDirectories?: string[],
  enabledToolNames?: Set<string>,
): Promise<string[]> {
  const prompts = [...existingSystemPrompt];

  // 1. 添加工作目录注释（无 emoji，使用绝对路径）
  if (additionalWorkingDirectories?.length) {
    prompts.push(
      `Additional working directories: ${additionalWorkingDirectories.join(', ')}`
    );
  }

  // 2. 添加 DiscoverSkills 指导（若启用）
  if (isDiscoverSkillsEnabled()) {
    prompts.push(getDiscoverSkillsSystemPrompt());
  }

  // 3. 添加环境信息
  prompts.push(await computeEnvInfo());

  return prompts;
}
```

## 9.4 工具过滤逻辑

```
// 第 500-502 行
const resolvedTools = useExactTools
  ? availableTools  // 使用传入的精确工具列表
  : resolveAgentTools(
      agentDefinition,
      availableTools,
      isAsync,
    ).resolvedTools;  // 根据 Agent 定义解析工具列表

// resolveAgentTools 实现逻辑:
// 1. 若 agentDefinition.tools === ['*'] → 返回全部 availableTools
// 2. 否则 → 过滤为 agentDefinition.tools 中的工具
// 3. 若 isAsync → 额外应用 ASYNC_AGENT_ALLOWED_TOOLS 限制
```

## 9.5 核心 query 调用

```
// 第 748-756 行
for await (const message of query({
  messages: initialMessages,
  systemPrompt: agentSystemPrompt,
  userContext: resolvedUserContext,
  systemContext: resolvedSystemContext,
  canUseTool,
  toolUseContext: agentToolUseContext,
  querySource,
  maxTurns: maxTurns ?? agentDefinition.maxTurns,
})) {
  // 处理每个消息事件
}
```

## 9.6 进度追踪与 TTFT 指标

```
// 第 758-768 行
onQueryProgress?.();  // 通知父级进度更新

// 转发 TTFT (Time to First Token) 指标给父 Agent
if (message.type === 'stream_event' &&
    message.event.type === 'message_start' &&
    message.ttftMs != null) {
  toolUseContext.pushApiMetricsEntry?.(message.ttftMs);
  continue;
}
```

## 9.7 元数据写入

```
// 第 735-742 行
// 在 query 循环前: 写入 Agent 元数据到磁盘

void recordSidechainTranscript(initialMessages, agentId).catch(...);

void writeAgentMetadata(agentId, {
```

```
  agentType: agentDefinition.agentType,
  ...(worktreePath && { worktreePath }),
  ...(description && { description }),
}).catch(...);

// 追踪最后记录的 message UUID（用于 transcript 追加）
let lastRecordedUuid: UUID | null =
  initialMessages.at(-1)?.uuid ?? null;
```

---

# 10. LocalAgentTask - 异步 Agent 生命周期管理

## 10.1 完整状态结构

文件：`tasks/LocalAgentTask/LocalAgentTask.tsx`（第 116-148 行）

```
export type LocalAgentTaskState = TaskStateBase & {
  type: 'local_agent';
  agentId: string;
  prompt: string;
  selectedAgent?: AgentDefinition;
  agentType: string;
  model?: string;
  abortController?: AbortController;
  unregisterCleanup?: () => void;
  error?: string;
  result?: AgentToolResult;
  progress?: AgentProgress;
  retrieved: boolean;
  messages?: Message[];
  lastReportedToolCount: number;
  lastReportedTokenCount: number;
  isBackgrounded: boolean;
  pendingMessages: string[];
  retain: boolean;
  diskLoaded: boolean;
  evictAfter?: number;
};
```

## 10.2 异步 Agent 注册

文件：`tasks/LocalAgentTask/LocalAgentTask.tsx`（第 466-515 行）

```
export function registerAsyncAgent({
  agentId,
  description,
  prompt,
  selectedAgent,
  setAppState,
  parentAbortController,
  toolUseId,
}): LocalAgentTaskState {
  // 创建中止控制器（支持层级取消）
```

```
  const abortController = parentAbortController
    ? createChildAbortController(parentAbortController)
    : createAbortController();

  const taskState: LocalAgentTaskState = {
    ...createTaskStateBase(agentId, 'local_agent', description, toolUseId),
    type: 'local_agent',
    status: 'running',
    agentId,
    prompt,
    selectedAgent,
    agentType: selectedAgent?.agentType ?? 'general-purpose',
    abortController,
    retrieved: false,
    isBackgrounded: true,    // 默认后台运行
    pendingMessages: [],
    retain: false,
    diskLoaded: false,
    lastReportedToolCount: 0,
    lastReportedTokenCount: 0,
  };

  // 注册到全局状态
  registerTask(taskState, setAppState);

  return taskState;
}
```

## 10.3 AbortController 层级

```
Parent AbortController (父 Agent)
    │
    ├── createChildAbortController()
    │         │
    │         ▼
    │   Child AbortController (子 Agent)
    │         │
    │         ├── 子 Abort → 取消子 Agent，不取消父 Agent
    │         └── 父 Abort → 自动取消所有子 Agent
    │
    └── 子 Agent 进程 IPC 通信
          └ <task-notification> 事件
```

## 10.4 进度追踪器

```
// 第 50-104 行
export type ProgressTracker = {
  toolUseCount: number;                    // 工具调用次数
  latestInputTokens: number;              // 最新输入 Token 数
  cumulativeOutputTokens: number;         // 累计输出 Token 数
  recentActivities: ToolActivity[];       // 最近活动记录
};
```

```
export function updateProgressFromMessage(
  tracker: ProgressTracker,
  message: Message,
  resolveActivityDescription?,
  tools?,
): void {
  // 更新最新输入 Token 数（包含 cache 读计数）
  tracker.latestInputTokens =
    usage.input_tokens +
    (usage.cache_creation_input_tokens ?? 0) +
    (usage.cache_read_input_tokens ?? 0);

  // 累加输出 Token
  tracker.cumulativeOutputTokens += usage.output_tokens;

  // 追踪工具调用（跳过 SYNTHETIC_OUTPUT_TOOL_NAME）
  tracker.toolUseCount += toolUseCount;
}
```

## 10.5 任务通知格式

文件：第 252-257 行

Agent 完成后，通过 `<task-notification>` XML 标签通知父 Agent：

```
<task-notification>
  <task-id>{taskId}</task-id>
  <status>completed</status>  <!-- completed | failed | killed -->
  <summary>{summary}</summary>
  <result>{finalMessage}</result>
  <usage>
    <total_tokens>N</total_tokens>
    <tool_uses>N</tool_uses>
    <duration_ms>N</duration_ms>
  </usage>
</task-notification>
```

> **Note**
>
> **CC中系统对子 agent 返回报告的内容没有任何结构化约束，但这可能造成agent交互过程中重要信息的丢失**

## 10.6 任务输出到磁盘

```
Agent 执行
    │
    ├── 写入元数据：~/.claude/agents/{agentId}/metadata.json
    ├── 写入 Transcript：~/.claude/agents/{agentId}/transcript.jsonl
    └── 输出文件路径：~/.claude/agents/{agentId}/output.txt

父 Agent 可通过 TaskOutputTool 读取 output.txt
```

# 11. RemoteAgent 与 InProcessTeammate

## 11.1 RemoteAgentTask

文件：`tasks/RemoteAgentTask/RemoteAgentTask.tsx`

远程 Agent 运行在 **CCR（Claude Code Remote）** 环境中：

```
// 启动条件：process.env.CLAUDE_CODE_REMOTE === "1"
async function runRemoteAgent() {
  // 1. 从环境变量获取 Agent 配置
  const agentConfig: AgentConfig = {
    agentId: process.env.CLAaude_CODE_AGENT_ID!,
    systemPrompt: process.env.CLAUDE_CODE_SYSTEM_PROMPT!,
    tools: JSON.parse(process.env.CLAUDE_CODE_TOOLS || "[]"),
  };

  // 2. 连接 MCP 服务器
  const mcpClient = await createMCPClient(process.env.MCP_SERVER_URL!);

  // 3. 注册 MCP 工具
  registerMcpTools(mcpClient);

  // 4. 运行 query 循环
  await runQueryWithMCP(agentConfig, mcpClient);
}
```

## 11.2 InProcessTeammateTask

文件：`tasks/InProcessTeammateTask/InProcessTeammateTask.tsx`

同进程内 Teammate，无子进程开销，共享内存：

```
// 特点：
// - 直接在主进程中运行（无需 IPC）
// - 共享 AppStateStore（直接修改状态）
// - 不能 spawn 其他后台 Agent
// - 通过内存队列接收 SendMessage

interface InProcessTeammateState {
  type: 'in_process_teammate';
  agentId: string;
  teamName: string;
  inbox: MessageQueue;  // 内存消息队列
}
```

**Note**

- 同进程 teammate 和父 agent 共享 AppState、文件权限、工具实例，但各自有独立的对话历史和身份上下文（通过 AsyncLocalStorage 隔离）
- 关于teammate 进行 sub-agent 的 spawn：
  - teammate 可以 spawn 普通 sub-agent（不带 name/team_name），但不能 spawn 另一个 teammate，并且不能spawn run_in_background: true 的子 agent。普通sub-agent指会阻塞主进程（主agent需要

等待）的那些，后台sub-agent则是主进程无需等待的那些

- 原因是：同进程 teammate 的生命周期由 leader 控制，它自己跑在 leader 进程里，**如果它再 spawn 一个后台异步子 agent，那个子 agent 的生命周期管理就会混乱**
- 发给 teammate 的 system prompt 里也明确写了：

```
- The name, team_name, and mode parameters are not available in this context
- teammates cannot spawn other teammates. Omit them to spawn a subagent.
```

---

## 12. StreamingToolExecutor - 工具并行编排引擎

**Note**

工具并行调用的好处：

- 延迟降低（省时）
- token效率提高，减少多轮 turn，减少历史消息积累（省钱）

### 12.1 核心设计原则

文件：`services/tools/StreamingToolExecutor.ts`

```
/**
 * 工具并行编排的核心设计：
 * - 并发安全（Concurrency-Safe）工具：可与其他并发安全工具并行执行
 * - 非并发安全工具：独占执行（必须单独运行）
 * - 结果按原始顺序缓冲和发出
 * - 流式执行：工具结果实时流回 API（支持多轮工具调用）
 */
export class StreamingToolExecutor { ... }
```

### 12.2 核心数据结构

```
type ToolStatus = 'queued' | 'executing' | 'completed' | 'yielded';

type TrackedTool = {
  id: string;                              // 工具调用 ID
  block: ToolUseBlock;                     // API 返回的工具调用块
  assistantMessage: AssistantMessage;      // 所属的 assistant 消息
  status: ToolStatus;                      // 当前状态
  isConcurrencySafe: boolean;              // 并发安全标志
  promise?: Promise<void>;                 // 执行 Promise
  results?: Message[];                     // 最终结果消息
  pendingProgress: Message[];              // 实时进度消息（立即 yield）
  contextModifiers?: Array<
    (context: ToolUseContext) => ToolUseContext  // 上下文修改器
  >;
};
```

注意：contextModifier 是工具执行后回调给执行引擎的补丁，用来动态调整后续工具的运行环境（权限、状态、工具池），而不是给 LLM 的额外消息

| | tool.results | contextModifiers |
|---|---|---|
| **修改对象** | 对话消息历史（给 LLM 看） | ToolUseContext（给**执行引擎**看） |
| **作用时机** | 随消息一起发给 API | 执行引擎内部消化，不发到 API |
| **效果** | LLM 知道工具输出什么 | 后续工具的执行行为被改变 |
| **典型用途** | 工具返回的文字/错误信息 | 工具执行后调整权限、状态、工具池 |

## 12.3 并发安全性判断

```
// 第 104-113 行
const parsedInput = toolDefinition.inputSchema.safeParse(block.input);
const isConcurrencySafe = parsedInput?.success
  ? (() => {
      try {
        // 调用工具定义的 isConcurrencySafe 回调
        return Boolean(toolDefinition.isConcurrencySafe(parsedInput.data));
      } catch {
        return false;  // 解析失败 → 不安全
      }
    })()
  : false;

// 默认安全性规则:
// - Read / Glob / Grep → 并发安全（无副作用）
// - Write / Edit / Bash → 非并发安全（有副作用）
```

**Note**

- 不分析工具间实际依赖，只按"是否可能产生副作用"一刀切
- 简单不精确，但保证正确
- 例如"工具B 可能会写文件，不确定它读什么"，那么采取保守策略：只要可能产生副作用，一律串行

## 12.4 执行队列算法

文件：第 140-150 行

```
private async processQueue(): Promise<void> {
  for (const tool of this.tools) {
    if (tool.status !== 'queued') continue;

    if (this.canExecuteTool(tool.isConcurrencySafe)) {
      // 条件满足: 开始执行
      await this.executeTool(tool);
    } else {
      // 条件不满足
      if (!tool.isConcurrencySafe) {
        // 非并发安全工具遇到冲突: 停止扫描（等待冲突工具完成）
        break;
      }
      // 并发安全工具但 slot 满了: 跳过（后面有空位时再处理）
```

```
    }
  }
}

// 决定是否可以执行
private canExecuteTool(isConcurrencySafe: boolean): boolean {
  const executingNonConcurrent = this.tools.some(
    t => t.status === 'executing' && !t.isConcurrencySafe
  );

  if (executingNonConcurrent) return false;  // 有独占工具在运行
  if (!isConcurrencySafe) return true;        // 非并发工具永远可以开始（独占）
  return true;                               // 并发安全工具可以并行
}
```

## 12.5 单工具执行流程

```
private async executeTool(tool: TrackedTool): Promise<void> {
  tool.status = 'executing';

  const toolDefinition = findToolByName(this.toolDefinitions, tool.block.name);

  if (!toolDefinition) {
    // 工具不存在：返回错误
    tool.results = [createUserMessage({
      content: [{
        type: 'tool_result',
        content: `<tool_use_error>Error: No such tool available: ${block.name}
</tool_use_error>`,
        is_error: true,
        tool_use_id: block.id,
      }],
      toolUseResult: `Error: No such tool available: ${block.name}`,
      sourceToolAssistantUUID: tool.assistantMessage.uuid,
    })];
    tool.status = 'completed';
    return;
  }

  // 权限检查
  if (!this.canUseTool(toolDefinition, tool.block.input)) {
    tool.results = [createUserMessage({
      content: [{
        type: 'tool_result',
        content: REJECT_MESSAGE,
        is_error: true,
        tool_use_id: block.id,
      }],
      toolUseResult: REJECT_MESSAGE,
      sourceToolAssistantUUID: tool.assistantMessage.uuid,
    })];
    tool.status = 'completed';
    return;
```

```
  }

  // 执行工具（创建子 AbortController）
  const toolAbortController = createChildAbortController(
    this.siblingAbortController
  );

  // 流式执行或普通执行
  if (toolDefinition.stream) {
    // 流式工具：实时 yield 进度
    tool.results = await this.streamTool(
      toolDefinition,
      tool.block.input,
      toolAbortController,
      tool,
    );
  } else {
    // 普通工具：直接调用
    tool.results = [await toolDefinition.invoke(
      tool.block.input,
      { ...this.toolUseContext, abortController: toolAbortController }
    )];
  }

  tool.status = 'completed';
}
```

## 12.6 错误传播与兄弟进程中止

```
// 第 359-363 行：Bash 工具出错时中止所有兄弟进程（同一轮 API 响应中并行发出的多个工具调用）
// 原因：Bash 错误通常意味着环境被破坏，兄弟进程的结果不可信

private siblingAbortController: AbortController;  // 子级 AbortController

// Bash 工具执行出错：
this.hasErrored = true;
this.erroredToolDescription = toolDescription;
// 中止所有兄弟进程（但不中止父级 query）
this.siblingAbortController.abort('sibling_error');

// 其他工具收到中止信号：
if (toolAbortController.signal.aborted) {
  tool.results = [createSyntheticErrorMessage('sibling_error')];
  return;
}
```

原因：Bash 报错通常意味着：环境被破坏了
父级不被中止，给主 agent 留有决策空间：父级 query loop 继续运行 ，主 agent 收到 3 个结果，决定报错、修复、或继续

## 12.7 结果聚合

```
// getCompletedResults: 按原始顺序 yield 完成的结果
async function* getRemainingResults(): AsyncGenerator<MessageUpdate> {
  while (true) {
    if (!this.hasPendingResults()) {
      // 无待处理结果：等待（progressAvailableResolve 唤醒）
      await new Promise<void>(r => { this.progressAvailableResolve = r; });
    }

    for (const tool of this.tools) {
      if (tool.status === 'completed' && !tool.yielded) {
        // 立即 yield 进度消息（实时性）
        for (const msg of tool.pendingProgress) {
          yield { message: msg };
        }

        // yield 最终结果
        for (const msg of tool.results ?? []) {
          yield { message: msg };
        }

        tool.yielded = true;
      }
    }
  }
}
```

---

# 13. 多 Agent 协调模式（Coordinator）

## 13.1 启用条件

文件：`coordinator/coordinatorMode.ts`

```
export function isCoordinatorMode(): boolean {
  if (feature('COORDINATOR_MODE')) {  // Statsig Feature Gate
    return isEnvTruthy(process.env.CLAUDE_CODE_COORDINATOR_MODE);
  }
  return false;
}
```

## 13.2 Coordinator 系统提示词（完整结构）

> **Note**
>
> - coordinator本身就是主session的主agent
> - 是否使用coordinator是启动时决定的，使用普通模式则进入普通设置，coordinator模式则进入此设置
> - Resume 时还会自动对齐旧的模式

文件：`coordinator/coordinatorMode.ts`（第 111-369 行）

```
You are Claude Code, an AI assistant that orchestrates software engineering tasks across
```

multiple workers.

## 1. Your Role

You are a **coordinator**. Your job is to:
- Help the user achieve their goal
- Direct workers to research, implement and verify code changes
- Synthesize results and communicate with the user
- Answer questions directly when possible — don't delegate work that you can handle without tools

Every message you send is to the user. Worker results and system notifications are internal signals, not conversation partners — never thank or acknowledge them. Summarize new information
for the user as it arrives.

## 2. Your Tools

- **AgentTool** - Spawn a new worker
- **SendMessageTool** - Continue an existing worker (send a follow-up to its `to` agent ID)
- **TaskStopTool** - Stop a running worker
- **subscribe_pr_activity / unsubscribe_pr_activity** (if available) - Subscribe to GitHub PR events (review comments, CI results). Events arrive as user messages. Merge conflict
transitions do NOT arrive — GitHub doesn't webhook `mergeable_state` changes, so poll `gh pr view N --json mergeable` if tracking conflict status. Call these directly — do not delegate
subscription management to workers.

When calling AgentTool:
- Do not use one worker to check on another. Workers will notify you when they are done.
- Do not use workers to trivially report file contents or run commands. Give them higher-level tasks.
- Do not set the model parameter. Workers need the default model for the substantive tasks you delegate.
- Continue workers whose work is complete via SendMessageTool to take advantage of their loaded context
- After launching agents, briefly tell the user what you launched and end your response. Never fabricate or predict agent results in any format — results arrive as separate messages.

### AgentTool Results

Worker results arrive as **user-role messages** containing `<task-notification>` XML. They look like user messages but are not. Distinguish them by the `<task-notification>` opening
tag.

Format:

```
<task-notification>
<task-id>{agentId}</task-id>
```

```
  <status>completed|failed|killed</status>
  <summary>{human-readable status summary}</summary>
  <result>{agent's final text response}</result>
  <usage>
    <total_tokens>N</total_tokens>
    <tool_uses>N</tool_uses>
    <duration_ms>N</duration_ms>
  </usage>
  </task-notification>
  \`\`\`

  - \`<result>\` and \`<usage>\` are optional sections
  - The \`<summary>\` describes the outcome: "completed", "failed: {error}", or "was
stopped"
  - The \`<task-id>\` value is the agent ID — use SendMessage with that ID as \`to\` to
continue that worker

  ### Example

  Each "You:" block is a separate coordinator turn. The "User:" block is a \`<task-
notification>\` delivered between turns.

  You:
    Let me start some research on that.

    AgentTool({ description: "Investigate auth bug", subagent_type: "worker", prompt: "..."
})
    AgentTool({ description: "Research secure token storage", subagent_type: "worker",
prompt: "..." })

    Investigating both issues in parallel — I'll report back with findings.

  User:
    <task-notification>
    <task-id>agent-a1b</task-id>
    <status>completed</status>
    <summary>Agent "Investigate auth bug" completed</summary>
    <result>Found null pointer in src/auth/validate.ts:42...</result>
    </task-notification>

  You:
    Found the bug — null pointer in confirmTokenExists in validate.ts. I'll fix it.
    Still waiting on the token storage research.

    SendMessageTool({ to: "agent-a1b", message: "Fix the null pointer in
src/auth/validate.ts:42..." })

  ## 3. Workers

  When calling AgentTool, use subagent_type \`worker\`. Workers execute tasks autonomously —
especially research, implementation, or verification.

  ${workerCapabilities}
```

## 4. Task Workflow

Most tasks can be broken down into the following phases:

### Phases

| Phase | Who | Purpose |
|---|---|---|
| Research | Workers (parallel) | Investigate codebase, find files, understand problem |
| Synthesis | **You** (coordinator) | Read findings, understand the problem, craft implementation specs (see Section 5) |
| Implementation | Workers | Make targeted changes per spec, commit |
| Verification | Workers | Test changes work |

### Concurrency

**Parallelism is your superpower. Workers are async. Launch independent workers concurrently whenever possible — don't serialize work that can run simultaneously and look for opportunities to fan out. When doing research, cover multiple angles. To launch workers in parallel, make multiple tool calls in a single message.**

Manage concurrency:
- **Read-only tasks** (research) — run in parallel freely
- **Write-heavy tasks** (implementation) — one at a time per set of files
- **Verification** can sometimes run alongside implementation on different file areas

### What Real Verification Looks Like

Verification means **proving the code works**, not confirming it exists. A verifier that rubber-stamps weak work undermines everything.

- Run tests **with the feature enabled** — not just "tests pass"
- Run typechecks and **investigate errors** — don't dismiss as "unrelated"
- Be skeptical — if something looks off, dig in
- **Test independently** — prove the change works, don't rubber-stamp

### Handling Worker Failures

When a worker reports failure (tests failed, build errors, file not found):
- Continue the same worker with SendMessageTool — it has the full error context
- If a correction attempt fails, try a different approach or report to the user

### Stopping Workers

Use TaskStopTool to stop a worker you sent in the wrong direction — for example, when you realize mid-flight that the approach is wrong, or the user changes requirements after you launched the worker. Pass the \`task_id\` from the AgentTool tool's launch result. Stopped workers can be continued with SendMessageTool.

## 5. Writing Worker Prompts

**Workers can't see your conversation.** Every prompt must be self-contained with everything the worker needs. After research completes, you always do two things: (1) synthesize findings into a specific prompt, and (2) choose whether to continue that worker via SendMessageTool or spawn a fresh one.

### Always synthesize — your most important job

When workers report research findings, **you must understand them before directing follow-up work**. Read the findings. Identify the approach. Then write a prompt that proves you understood by including specific file paths, line numbers, and exactly what to change.

Never write "based on your findings" or "based on the research." These phrases delegate understanding to the worker instead of doing it yourself. You never hand off understanding to another worker.

### Continue vs. spawn by context overlap

| Situation | Mechanism | Why |
|---|---|---|
| Research explored exactly the files that need editing | **Continue** (SendMessageTool) | Worker already has the files in context AND now gets a clear plan |
| Research was broad but implementation is narrow | **Spawn fresh** (AgentTool) | Avoid dragging along exploration noise; focused context is cleaner |
| Correcting a failure or extending recent work | **Continue** | Worker has the error context and knows what it just tried |
| Verifying code a different worker just wrote | **Spawn fresh** | Verifier should see the code with fresh eyes |
| First implementation attempt used wrong approach | **Spawn fresh** | Wrong-approach context pollutes the retry; clean slate avoids anchoring on failed path |
| Completely unrelated task | **Spawn fresh** | No useful context to reuse |

### Prompt tips

**Good examples:**

1. Implementation: "Fix the null pointer in src/auth/validate.ts:42. The user field can be undefined when the session expires. Add a null check and return early with an appropriate error. Commit and report the hash."

2. Precise git operation: "Create a new branch from main called 'fix/session-expiry'. Cherry-pick only commit abc123 onto it. Push and create a draft PR targeting main. Add anthropics/claude-code as reviewer. Report the PR URL."

3. Correction (continued worker, short): "The tests failed on the null check you added — validate.test.ts:58 expects 'Invalid session' but you changed it to 'Session expired'. Fix the assertion. Commit and report the hash."

**Bad examples:**

```
  4. "Fix the bug we discussed" — no context, workers can't see your conversation
  5. "Based on your findings, implement the fix" — lazy delegation
  6. "Create a PR for the recent changes" — ambiguous scope
  7. "Something went wrong with the tests, can you look?" — no error, no path, no direction

  ## 6. Example Session

  User: "There's a null pointer in the auth module. Can you fix it?"

  You:
    Let me investigate first.

    AgentTool({ description: "Investigate auth bug", subagent_type: "worker", prompt:
"Investigate the auth module in src/auth/. Find where null pointer exceptions could occur
around session
   handling and token validation... Report specific file paths, line numbers, and types
involved. Do not modify files." })
    AgentTool({ description: "Research auth tests", subagent_type: "worker", prompt: "Find
all test files related to src/auth/. Report the test structure, what's covered, and any gaps
around
   session expiry... Do not modify files." })

    Investigating from two angles — I'll report back with findings.

  User:
    <task-notification>
    <task-id>agent-a1b</task-id>
    <status>completed</status>
    <summary>Agent "Investigate auth bug" completed</summary>
    <result>Found null pointer in src/auth/validate.ts:42...</result>
    </task-notification>

  You:
    Found the bug — null pointer in validate.ts:42.
    SendMessageTool({ to: "agent-a1b", message: "Fix the null pointer in
src/auth/validate.ts:42. Add a null check before accessing user.id — if null, return 401
with 'Session expired'.
  Commit and report the hash." })

    Fix is in progress.
```

---

**系统提示词包含以下部分：**

1. **角色定义**：协调器，管理多个软件工程 Worker
2. **可用工具**：
   - `Agent` - Spawn 新 Worker
   - `SendMessage` - 继续已有 Worker
   - `TaskStop` - 停止 Worker
   - `subscribe_pr_activity` / `unsubscribe_pr_activity` - GitHub PR 事件订阅

3. **Worker 能力**：

```
// 简单模式（CLAUDE_CODE_SIMPLE=1）：
// Workers have access to Bash, Read, and Edit tools, plus MCP tools...

// 完整模式：
// Workers have access to standard tools, MCP tools,
// and project skills via the Skill tool...
```

4. **并行策略**：
   - 研究任务：自由并行（Read-only）
   - 实现任务：同一文件组一次一个 Worker
5. **通信协议**：通过 `<task-notification>` XML 标签接收 Worker 结果
6. **继续 vs. Spawn 决策矩阵**：
   - 高上下文重叠 → `SendMessage` 继续
   - 低上下文重叠 / 方法错误 / 无关任务 → Spawn 新 Worker
7. **工作流**：
   - 研究：Worker（并行）
   - 综合：Coordinator
   - 实现：Worker
   - 验证：Worker

---

**两种模式的提示词对比**：

1. 角色定义

普通模式：通用 agent，自己完成所有任务，直接调用 Read / Write / Bash 等工具。
Coordinator 模式：协调者角色，自己不执行，只调度。禁止直接操作文件，全部通过 AgentTool / SendMessageTool / TaskStopTool 与 workers 通信。

2. 工具层

普通模式拥有完整工具池（Read/Write/Bash/Grep/WebFetch 等），工具调用直接执行。
Coordinator 模式工具池极度受限，仅 AgentTool、SendMessageTool、TaskStopTool 三个工具。工具调用全部委托给 worker agents，主 agent 本身不碰项目文件。

3. 并发模型

普通模式是工具级并行，通过 isConcurrencySafe 自动调度多个工具并行执行（Read/Glob/Grep 可并行，Bash/Write/Edit 串行）。
Coordinator 模式是任务级并行，主 agent 主动设计并行策略：读任务（research）并行，写任务（implementation）串行。核心原则是 "Parallelism is your superpower. Workers are async. Launch independent workers concurrently."

4. 任务生命周期

普通模式是线性执行：用户请求 -> 主 agent 直接完成 -> 报告结果。
Coordinator 模式是四阶段流水线：

- Phase 1 Research：Workers 并行研究代码库、理解问题
- Phase 2 Synthesis：Coordinator 消化研究结果，理解问题，写出精确实现规范（含文件路径、行号）
- Phase 3 Implementation：Workers 按规范实现，提交代码

- Phase 4 Verification：Workers 独立对抗性验证

5. 通信模型

普通模式消息直接来自用户，主 agent 直接回应。
Coordinator 模式主 agent 收到的消息大部分是 `<task-notification>`（worker 的结果）而非真实用户输入。
Worker results and system notifications are internal signals, not conversation partners — never thank or acknowledge them。

6. 验证方式

普通模式主 agent 可自己 + 可选 verification subagent 共同验证，允许 self-assign PASS/FAIL。
Coordinator 模式验证由独立 worker 负责，only the verifier assigns a verdict，主 agent 禁止自我赋值。
Verification means proving the code works, not confirming it exists。

7. 反模式约束

Coordinator prompt 专门列出必须避免的 delegation 坏味道：

- "Fix the bug we discussed" — workers 看不见对话，无上下文
- "Based on your findings..." — 把理解责任推给 worker，不自己做
- "Create a PR for the recent changes" — 模糊：哪些变更？哪个分支？
- "Something went wrong, can you look?" — 无错误信息、无路径、无方向
  普通模式没有这种专门的 delegation 反模式列表。

8. 错误恢复策略

普通模式失败时重试、换方案或问用户。
Coordinator 模式失败时继续同一个 worker（SendMessageTool），因为它有完整错误上下文。If a correction attempt fails, try a different approach or report to the user。

## 13.3 Coordinator 工具上下文

```
// getCoordinatorUserContext（第 80 行起）
export function getCoordinatorUserContext(
  mcpClients: ReadonlyArray<{ name: string }>,
  scratchpadDir?: string,  // Statsig Feature Gate: tengu_scratch
): { [k: string]: string } {
  // 构建 workerToolsContext
  const workerTools = isEnvTruthy(process.env.CLAUDE_CODE_SIMPLE)
    ? [BASH_TOOL_NAME, FILE_READ_TOOL_NAME, FILE_EDIT_TOOL_NAME].sort().join(', ')
    : Array.from(ASYNC_AGENT_ALLOWED_TOOLS)
        .filter(name => !INTERNAL_WORKER_TOOLS.has(name))  // 排除内部工具
        .sort().join(', ');

  return { workerToolsContext: workerToolsPrompt };
}
```

## 13.4 Worker 工具白名单

文件：`constants/tools.ts`

```
// Coordinator Worker 可使用的工具（ASYNC_AGENT_ALLOWED_TOOLS）
// AgentTool 内部使用：仅允许这些工具通过 AgentTool spawn
```

```
export const ASYNC_AGENT_ALLOWED_TOOLS: ReadonlySet<string> = new Set([
    FILE_READ_TOOL_NAME,        // Read
    WEB_SEARCH_TOOL_NAME,      // WebSearch
    TODO_WRITE_TOOL_NAME,       // TodoWrite
    GREP_TOOL_NAME,             // Grep
    WEB_FETCH_TOOL_NAME,        // WebFetch
    GLOB_TOOL_NAME,             // Glob
    ...SHELL_TOOL_NAMES,        // Bash, PowerShell
    FILE_EDIT_TOOL_NAME,        // Edit
    FILE_WRITE_TOOL_NAME,       // Write
    NOTEBOOK_EDIT_TOOL_NAME,    // NotebookEdit
    SKILL_TOOL_NAME,            // Skill
    SYNTHETIC_OUTPUT_TOOL_NAME,  // SyntheticOutput
    TOOL_SEARCH_TOOL_NAME,      // ToolSearch
    ENTER_WORKTREE_TOOL_NAME,    // EnterWorktree
    EXIT_WORKTREE_TOOL_NAME,     // ExitWorktree
]);
```

**IN_PROCESS_TEAMMATE_ALLOWED_TOOLS（同进程 teammate 额外可用）**

```
export const IN_PROCESS_TEAMMATE_ALLOWED_TOOLS = new Set([
    TASK_CREATE_TOOL_NAME,   // TaskCreate
    TASK_GET_TOOL_NAME,      // TaskGet
    TASK_LIST_TOOL_NAME,     // TaskList
    TASK_UPDATE_TOOL_NAME,   // TaskUpdate
    SEND_MESSAGE_TOOL_NAME,  // SendMessage
    // 条件启用（AGENT_TRIGGERS feature gate）:
    ...(feature('AGENT_TRIGGERS')
      ? [CRON_CREATE_TOOL_NAME, CRON_DELETE_TOOL_NAME, CRON_LIST_TOOL_NAME]
      : []),
  ])
```

### 13.5 内部工具（Worker 不可见）

```
// Coordinator 专用工具，对 Worker 不可见
const INTERNAL_WORKER_TOOLS = new Set([
  TEAM_CREATE_TOOL_NAME,    // TeamCreateTool
  TEAM_DELETE_TOOL_NAME,    // TeamDeleteTool
  SEND_MESSAGE_TOOL_NAME,   // SendMessageTool
  SYNTHETIC_OUTPUT_TOOL_NAME, // SyntheticOutputTool
]);
```

---

## 14. 上下文压缩系统（Compaction）

> **Note**
>
> 详见 上下文管理 对应的文档

### 14.1 压缩策略体系

Claude Code 实现了**三级压缩体系**：

```
Token 预算
    │
    ├── 正常区域（< AUTOCOMPACT_THRESHOLD）
    │      └─ 无压缩，正常运行
    │
    ├── 警告区域（> WARNING_THRESHOLD）
    │      └─ 显示警告提示
    │
    ├── Auto Compact 触发点（> AUTOCOMPACT_THRESHOLD）
    │      └─ 自动触发压缩
    │
    └── 手动压缩区域（/compact 命令）
           ├─ snip 模式：仅截断
           ├─ full 模式：完整压缩
           └─ aggressive 模式：激进摘要
```

## 14.2 精确阈值

文件：`services/compact/autoCompact.ts`

```
// 压缩摘要输出的最大 Token 数
// 基于 p99.99 压缩摘要输出为 17,387 Token 的观测
const MAX_OUTPUT_TOKENS_FOR_SUMMARY = 20_000;

// Auto Compact 触发阈值缓冲
export const AUTOCOMPACT_BUFFER_TOKENS = 13_000;

// 警告阈值缓冲
export const WARNING_THRESHOLD_BUFFER_TOKENS = 20_000;

// 错误阈值缓冲
export const ERROR_THRESHOLD_BUFFER_TOKENS = 20_000;

// 手动压缩缓冲
export const MANUAL_COMPACT_BUFFER_TOKENS = 3_000;

// 自动压缩失败熔断阈值
const MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3;
// 注：2026-03-10 数据显示，每天约 250K 次 API 调用因连续压缩失败而浪费
```

## 14.3 自动压缩阈值计算

```
// 第 72-91 行
export function getAutoCompactThreshold(model: string): number {
  // 有效上下文窗口 = 模型上下文窗口 - 保留给摘要输出的 Token
  const effectiveContextWindow = getEffectiveContextWindowSize(model);

  let autocompactThreshold =
    effectiveContextWindow - AUTOCOMPACT_BUFFER_TOKENS;
  // = effectiveContextWindow - 13,000

  // 环境变量覆盖（用于测试）
```

```
  const envPercent = process.env.CLAUDE_AUTOCOMPACT_PCT_OVERRIDE;
  if (envPercent) {
    const parsed = parseFloat(envPercent);
    if (!isNaN(parsed) && parsed > 0 && parsed <= 100) {
      return Math.floor(effectiveContextWindow * (parsed / 100));
    }
  }

  return autocompactThreshold;
}

// effectiveContextWindow 计算（第 33-49 行）
export function getEffectiveContextWindowSize(model: string): number {
  const reservedTokensForSummary = Math.min(
    getMaxOutputTokensForModel(model),
    MAX_OUTPUT_TOKENS_FOR_SUMMARY,  // 20,000
  );

  let contextWindow = getContextWindowForModel(model, getSdkBetas());

  // 环境变量覆盖
  if (process.env.CLAUDE_CODE_AUTO_COMPACT_WINDOW) {
    const parsed = parseInt(process.env.CLAUDE_CODE_AUTO_COMPACT_WINDOW, 10);
    if (!isNaN(parsed) && parsed > 0) {
      contextWindow = Math.min(contextWindow, parsed);
    }
  }

  return contextWindow - reservedTokensForSummary;
}
```

## 14.4 压缩执行算法

文件：`services/compact/compact.ts`

```
export async function compactConversation(
  messages: Message[],
  options: CompactOptions,
): Promise<CompactionResult> {
  // 1. 识别可压缩的起始点（pivot index）
  const pivotIndex = findPivotIndex(messages);

  // 2. 对 pivot 之前的消息生成摘要
  const summaryPrompt = buildSummaryPrompt(messages.slice(0, pivotIndex));
  const summaryMessage = await callSummaryModel(summaryPrompt);

  // 3. 构建压缩后的消息列表
  const compressedMessages = [
    ...messages.slice(0, pivotIndex).map(m => markAsCompacted(m)),
    createSummaryMessage(summaryMessage),
    ...messages.slice(pivotIndex),
  ];
```

```
  // 4. 恢复最近的附件文件（最多 5 个，共 5,000 tokens）
  const restoredFiles = restoreRecentAttachments(
    compressedMessages,
    { maxFiles: 5, maxTokensPerFile: 5_000 }
  );

  // 5. 恢复技能信息（5,000 tokens/skill，总共 25,000 tokens）
  const restoredSkills = restoreSkillInfo(
    compressedMessages,
    { maxSkills: 5, maxTokensPerSkill: 5_000 }
  );

  return {
    messages: compressedMessages,
    restoredFiles,
    restoredSkills,
    pivotIndex,
    summaryMessage,
  };
}
```

## 14.5 压缩后的墓碑机制

```
// 压缩后的消息被标记为墓碑（TombstoneMessage）
// 显示为灰色占位符，不计入 Token
interface TombstoneMessage {
  type: 'tombstone';
  originalCount: number;  // 被压缩的消息数量
  summaryId: string;      // 摘要 ID（可用于展开）
}

// 压缩边界消息
interface CompactBoundaryMessage {
  type: 'compact_boundary';
  pivotUuid: string;
  messageCountBefore: number;
  tokensSaved: number;
}
```

## 14.6 压缩熔断器（Circuit Breaker）

```
// 连续 3 次压缩失败后停止重试
// 防止不可恢复的上下文超限反复浪费 API 调用

export async function autoCompactIfNeeded(
  messages: Message[],
  tracking?: AutoCompactTrackingState,
): Promise<{ wasCompacted: boolean; consecutiveFailures?: number }> {
  // 检查是否超过阈值
  const currentTokens = tokenCountWithEstimation(messages);
  const threshold = getAutoCompactThreshold(currentModel);
```

```
  if (currentTokens <= threshold) {
    return { wasCompacted: false };
  }

  try {
    // 尝试压缩
    const result = await compactConversation(messages, options);
    return { wasCompacted: true, compactionResult: result };
  } catch (error) {
    const consecutiveFailures = (tracking?.consecutiveFailures ?? 0) + 1;

    if (consecutiveFailures >= MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES) {
      // 熔断：停止重试
      logError(`Auto-compact failed ${consecutiveFailures} times, circuit breaker
triggered`);
      return { wasCompacted: false, consecutiveFailures };
    }

    // 继续重试
    return { wasCompacted: false, consecutiveFailures };
  }
}
```

### 14.7 压缩触发的 transcript 刷写

```
// query.ts 中，压缩边界前刷写内存 transcript
// 第 701-715 行
if (compactBoundaryIndex !== undefined) {
  // 在 compact boundary 消息写入前：
  // 1. 刷入内存中的消息到 transcript
  const tailIdx = findTailIndex(state.messages, compactBoundaryIndex);
  await recordTranscript(state.messages.slice(0, tailIdx + 1));
}
```

---

## 15. 记忆系统（Memory）

> **Note**
>
> 详见 记忆系统 对应的文档

### 15.1 记忆类型

文件：`memdir/memdir.ts`

```
export const MEMORY_TYPES = ['user', 'feedback', 'project', 'reference'] as const;
export type MemoryType = (typeof MEMORY_TYPES)[number];
```

| 类型 | 内容 | 保存时机 |
|---|---|---|
| `user` | 用户角色、目标、偏好 | 学习到用户背景时 |
| `feedback` | 用户的指导（避免/坚持的做法） | 用户纠正时 |

| 类型 | 内容 | 保存时机 |
| --- | --- | --- |
| `project` | 项目状态、目标、已知 bug | 项目上下文变化时 |
| `reference` | 外部系统引用（URL、文档位置） | 发现外部信息时 |

## 15.2 记忆目录结构

```
~/.claude/projects/{slug}/memory/
    │
    ├── MEMORY.md          # 索引文件（最多 200 行，25KB）
    │                      # 格式：`- [Title](file.md) — one-line hook`
    │
    ├── user/
    │     └── user_role.md    # 用户信息
    │
    ├── feedback/
    │     └── feedback_*.md   # 反馈记录
    │
    ├── project/
    │     └── project_*.md    # 项目上下文
    │
    └── reference/
          └── reference_*.md  # 外部引用
```

## 15.3 记忆文件格式

```
---
name: {{memory name}}
description: {{单行描述 — 用于判断相关性}}
type: {{user | feedback | project | reference}}
---

{{memory content — feedback/project 类型需包含 WHY 和 How to apply}}
```

**feedback 类型特殊格式：**

```
---
name: avoid_synthetic_mocks
description: 不要 mock 数据库，用真实集成测试
type: feedback
---

不要在集成测试中使用 mock 数据库 — **Why:** 上季度 mock 测试通过但生产迁移失败。**How to apply:** 所有测试必须 hit 真实数据库，使用 testcontainers 或内存数据库。
```

## 15.4 记忆加载流程

```
// memdir.ts
export async function loadMemoryPrompt(): Promise<string | null> {
  // 1. 读取 MEMORY.md 索引文件
  const memoryIndex = await readFile(memoryIndexPath, 'utf-8');
```

```
  // 2. 解析索引行
  const entries = parseMemoryIndex(memoryIndex);

  // 3. 过滤相关条目（基于当前查询）
  const relevantEntries = filterRelevantEntries(entries, currentQuery);

  // 4. 加载相关记忆文件内容
  const memoryContents = await Promise.all(
    relevantEntries.map(entry => readMemoryFile(entry.path))
  );

  // 5. 注入到系统提示词
  return buildMemoryInjectionPrompt(memoryContents);
}
```

## 15.5 记忆与系统提示词的关系

文件：`QueryEngine.ts`（第 286-325 行）

```
// 记忆作为系统提示词的动态部分注入
async function fetchSystemPromptParts() {
  const [defaultSystemPrompt, userContext, systemContext] = await Promise.all([
    getDefaultSystemPrompt(),           // 静态部分
    getUserContext(),                   // 用户信息
    getSystemContext(),                 // 系统信息
  ]);

  return {
    defaultSystemPrompt,
    userContext,
    systemContext,
  };
}

// MEMORY_AUTO_MEM_PATH: 自动记忆路径
// 用户可通过 CLAUDE_CODE_AUTO_MEM_PATH 环境变量指定
// 自定义 prompt 可以覆盖此路径
```

## 15.6 团队记忆（Team Memory）

文件：`memdir/teamMemPrompts.ts`

```
// Team 模式下的共享记忆
interface TeamMemory {
  teamId: string;
  sharedContext: MemoryEntry[];   // 团队共享上下文
  individualContext: {             // 各成员上下文
    [teammateId: string]: MemoryEntry[];
  };
}

// Team 记忆的特点:
```

```
// - 共享层：所有成员可见（项目目标、架构决策）
// - 个体层：各成员私有（各自的当前任务）
```

---

# 16. 权限系统（Permissions）

## 16.1 权限类型

```
type Permission =
  | 'bash'              // Shell 命令执行
  | 'bash-sudo'         // 特权 Shell 命令
  | 'edit'              // 文件编辑
  | 'write'             // 文件写入
  | 'read'              // 文件读取
  | 'network'           // 网络请求
  | 'clipboard'         // 剪贴板访问
  | 'run-agent'         // 启动子 Agent
  | 'use-mcp'           // 使用 MCP 服务器
  | 'web-search'        // 互联网搜索
  | 'web-fetch';        // 网页抓取
```

## 16.2 权限级别

```
enum PermissionLevel {
  DENY = 0,      // 明确拒绝
  PROMPT = 1,    // 每次询问
  ALLOW = 2,     // 允许
  TRUST = 3,     // 完全信任
}

// 权限规则匹配
interface PermissionRule {
  pattern: string | RegExp;  // 命令/文件路径匹配模式
  level: PermissionLevel;
}
```

## 16.3 权限检查流程

文件：`hooks/useCanUseTool.ts`

```
type CanUseToolFn = (
  tool: Tool,
  toolInput: unknown,
) => PermissionCheckResult;

interface PermissionCheckResult {
  allowed: boolean;
  reason?: string;
  behavior: 'allow' | 'deny' | 'prompt';
}

// 检查结果的行为
```

```
function checkPermission(
  permission: Permission,
  tool: Tool,
  toolInput: unknown,
): PermissionCheckResult {
  // 1. 获取配置的权限级别
  const level = getPermissionLevel(permission);

  switch (level) {
    case DENY:
      return { allowed: false, reason: 'denied_by_config', behavior: 'deny' };

    case TRUST:
      return { allowed: true, behavior: 'allow' };

    case ALLOW:
      return { allowed: true, behavior: 'allow' };

    case PROMPT:
      // 弹出对话框询问用户
      if (tool.allowedWhenPaused) {
        return { allowed: true, behavior: 'allow' };
      }
      return { allowed: false, reason: 'requires_user_confirmation', behavior: 'prompt' };
  }
}
```

## 16.4 Agent 权限模式

```
// AgentTool 支持自定义权限模式
type PermissionMode =
  | 'acceptEdits'     // 接受编辑（默认）
  | 'restricted'      // 受限（只读工具）
  | 'browseOnly'      // 仅浏览（无写入）
  | 'custom';        // 自定义规则

// Agent 定义可指定权限模式
interface AgentDefinition {
  permissionMode?: PermissionMode;
}
```

## 16.5 权限拒绝追踪

```
// QueryEngine.ts 中的权限追踪
// 用于向 SDK 调用者报告权限使用情况
class QueryEngine {
  private permissionDenials: SDKPermissionDenial[] = [];

  private wrappedCanUseTool(
    originalCanUseTool: CanUseToolFn,
  ): CanUseToolFn {
    return (tool, input) => {
```

```
      const result = originalCanUseTool(tool, input);
      if (result.behavior !== 'allow') {
        // 追踪拒绝/提示事件
        this.permissionDenials.push({
          toolName: tool.name,
          reason: result.reason ?? 'permission_check_failed',
          timestamp: Date.now(),
        });
      }
      return result;
    };
  }
}
```

---

# 17. 状态管理（AppStateStore）

## 17.1 完整状态结构

文件：`state/AppStateStore.ts`

```
export type AppState = DeepImmutable<{
  // ===== 配置 =====
  settings: SettingsJson;              // 用户配置
  verbose: boolean;                    // 详细模式
  mainLoopModel: ModelSetting;          // 主循环模型
  toolPermissionContext: ToolPermissionContext; // 工具权限上下文

  // ===== MCP =====
  mcp: {
    clients: MCPServerConnection[];    // 已连接的 MCP 服务器
    tools: Tool[];                     // MCP 工具（动态注册）
    commands: Command[];               // MCP 命令
    resources: Record<string, ServerResource[]>; // MCP 资源
    pluginReconnectKey: number;        // 重连键
  };

  // ===== 任务管理 =====
  tasks: { [taskId: string]: TaskState };  // 所有 Agent 任务
  agentNameRegistry: Map<string, AgentId>;  // 名称 → AgentId 路由
  foregroundedTaskId?: string;             // 前台任务
  viewingAgentTaskId?: string;             // 查看中的 Agent

  // ===== 文件历史 =====
  fileHistory: FileHistoryState;

  // ===== 任务清单 =====
  todos: { [agentId: string]: TodoList };

  // ===== 通知 =====
  notifications: {
    current: Notification | null;      // 当前通知
    queue: Notification[];             // 通知队列
```

```
  };

  // ===== 思考模式 =====
  thinkingEnabled: boolean | undefined;

  // ===== Session Hooks =====
  sessionHooks: SessionHooksState;

  // ===== Team 上下文 =====
  teamContext?: {
    teamName: string;
    teamFilePath: string;
    leadAgentId: string;
    selfAgentId?: string;
    teammates: { [teammateId: string]: TeammateState };
  };

  // ===== 消息收件箱 =====
  inbox: { messages: Array<InboxMessage> };

  // ===== 推测/预填充状态 =====
  speculation: SpeculationState;

  // ===== Coordinator =====
  coordinatorTaskIndex: number;
}>;
```

## 17.2 状态更新模式

```
// 使用函数式更新，避免直接修改
interface SetAppStateFn {
  (setter: (prev: AppState) => AppState): void;
}

// 示例：注册 Agent
function registerAsyncAgent(params, setAppState: SetAppStateFn) {
  setAppState(prev => ({
    ...prev,
    tasks: {
      ...prev.tasks,
      [taskId]: createTaskState(taskId, params),
    },
  }));
}

// 示例：注册 Agent 名称路由
setAppState(prev => {
  const next = new Map(prev.agentNameRegistry);
  next.set(name, agentId);
  return { ...prev, agentNameRegistry: next };
});
```

### 17.3 Team 上下文状态

```
// Team 模式下，teamContext 存储团队状态
interface TeamContext {
  teamName: string;           // Team 名称
  teamFilePath: string;       // Team 定义文件路径
  leadAgentId: string;        // 主控 Agent（Coordinator）
  selfAgentId?: string;       // 当前 Agent 的 ID
  teammates: {
    [teammateId: string]: {
      agentId: string;
      status: TaskStatus;
      inbox: Message[];
      lastSeen: number;
    };
  };
}
```

---

## 18. Slash 命令系统

> **Note**
>
> - Slash 命令是一种特殊的输入格式，用于触发**预定义的工作流**
> - 用户在输入框中输入 `/命令名 [参数]` 即可触发，例如 `/resume`、`/config`
> - **普通用户可自行定义 slash 命令**，只要在 .claude/skills/ 目录下写一个 markdown 文件即可

### 18.1 命令注册

文件：`commands/`（88+ 个命令）

```
interface Command {
  name: string;               // 命令名称（不含 /）
  description: string;         // 帮助描述
  aliases?: string[];          // 别名
  flags?: CommandFlag[];      // 支持的参数
  handler: CommandHandler;    // 处理函数（支持 async generator）
}

// 命令优先级（用于竞态处理）
type CommandPriority =
  | 'pause_after_next'
  | 'pause_immediately'
  | 'resume'
  | 'cancel_resume';
```

### 18.2 主要命令列表

| 命令 | 文件 | 功能 |
|---|---|---|
| `/commit` | `commands/commit/` | 生成 git commit |

| 命令 | 文件 | 功能 |
| --- | --- | --- |
| `/review` | `commands/review/` | 代码审查 |
| `/config` | `commands/config/` | 配置管理 |
| `/init` | `commands/init/` | 项目初始化 |
| `/help` | `commands/help/` | 帮助信息 |
| `/compact` | `commands/compact/` | 上下文压缩 |
| `/model` | `commands/model/` | 切换模型 |
| `/retry` | `commands/retry/` | 重试 |
| `/undo` | `commands/undo/` | 撤销 |
| `/skill` | `commands/skills/` | Skill 管理 |
| `/agent` | `commands/agents/` | Agent 管理 |
| `/exit` | `commands/exit/` | 退出 |
| `/prompt` | `commands/prompt/` | 自定义提示词 |
| `/attach` | `commands/attach/` | 附件处理 |

## 18.3 命令执行流程

```
用户输入 /compact --snip
    │
    ▼
InputParser.parse(input)
    │  解析为 { command: 'compact', args: ['--snip'] }
    ▼
CommandRegistry.get('compact')
    │  获取命令定义和优先级
    ▼
checkPriorityConflict()  // 检查与 pending 命令的优先级冲突
    │
    ▼
CommandLifecycleNotifier.notify('starting')
    │  通知生命周期事件
    ▼
executeCommand(handler, args, flags)
    │
    ├── 暂停当前 queryLoop
    ├── 执行命令 handler
    ├── 命令可能 yield 消息给 query
    └── 命令完成后恢复或终止 query
    │
    ▼
CommandLifecycleNotifier.notify('completed')
```

---

# 19. API 调用层

## 19.1 Claude API 客户端

文件：`services/api/claude.ts`

```
// API 版本
const ANTHROPIC_VERSION = '2023-06-01';

// 核心请求构建
interface ClaudeAPIParams {
  model: string;
  system: string;           // 已渲染的系统提示词
  messages: APIMessage[];  // 格式化后的消息
  max_tokens: number;
  temperature?: number;
  tools?: APIToolDefinition[];
  // Beta 特性
  betas?: string[];
  // 提示缓存
  cache_control?: CacheControl;
}

// 流式响应处理
async function* streamClaudeResponse(
  response: Response,
): AsyncGenerator<StreamEvent | AssistantMessage> {
  const reader = response.body?.getReader();
  const decoder = new TextDecoder();

  for await (const chunk of streamReader(reader, decoder)) {
    yield* parseSSEEvent(chunk);  // 解析 Server-Sent Events
  }
}
```

## 19.2 模型配置

文件：`utils/model/model.ts`

```
export function getRuntimeMainLoopModel(): string {
  // 1. 读取 settings.json 中的 model 设置
  // 2. 检查 CLAUDE_CODE_MODEL 环境变量
  // 3. 检查 --model 命令行参数
  // 4. 降级到默认模型（sonnet-4-5）
}

export function getDefaultSubagentModel(): string {
  // 子 Agent 默认模型（可能与主模型不同）
  // 通常比主模型更快/更便宜
  return 'sonnet-4-5';
}
```

## 19.3 Token 预算管理

文件：`query/tokenBudget.ts`

```
// 基于 feature('TOKEN_BUDGET') 启用
export function createBudgetTracker() {
  // 追踪每个 turn 的 Token 消耗
```

```
  // 当累积消耗达到阈值时触发自动 continue
  // 支持 +500,000 Token 的自动继续（beta feature: task-budgets-2026-03-13）
}

export function checkTokenBudget(
  usage: NonNullableUsage,
  budgetTracker: BudgetTracker,
): 'continue' | 'stop';
```

---

# 20. 关键算法流程详图

## 20.1 完整 Query Loop 流程

> **Note**
>
> 注意上下文压缩判断是在刚进入query loop时最早进行的

```
┌──────────────────────────────────────────────────────────────────────┐
│                        queryLoop() 开始                              │
│                                                                      │
│  State = {                                                           │
│    messages: params.messages,                                         │
│    toolUseContext: params.toolUseContext,                            │
│    autoCompactTracking: undefined,                                    │
│    maxOutputTokensRecoveryCount: 0,                                   │
│    hasAttemptedReactiveCompact: false,                               │
│    turnCount: 1,                                                      │
│    ...                                                                │
│  }                                                                   │
│                                                                      │
│  budgetTracker = createBudgetTracker()  // 若 TOKEN_BUDGET 开启    │
│  taskBudgetRemaining = undefined                                     │
│  config = buildQueryConfig()  // 快照不可变配置                    │
└──────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────┐
│                  while (true) — 主循环                               │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │ ITERATION START: turnCount = N                                 │    │
│  │                                                                │    │
│  │ 1. compactIfNeeded()                                           │    │
│  │    ├─ tokenCountWithEstimation(messages)                       │    │
│  │    ├─ 若 > AUTOCOMPACT_BUFFER_TOKENS → autoCompact()          │    │
│  │    └─ 若 REACTIVE_COMPACT 开启 → 尝试 reactiveCompact()     │    │
│  │                                                                │    │
│  │ 2. buildQueryConfig()  // 仅首次迭代                          │    │
│  │                                                                │    │
│  │ 3. apiClient.createMessage({                                   │    │
│  │      model: config.model,                                      │    │
```

```
│   │      system: systemPrompt,                                    │   │
│   │      messages: normalizeMessagesForAPI(messages),             │   │
│   │      max_tokens: maxOutputTokensOverride,                    │   │
│   │      tools: buildToolDefinitions(config.tools),               │   │
│   │    })                                                         │   │
│   │                                                              │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                              │                                        │
│                              ▼                                        │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │ for await (const event of streamResponse)                    │   │
│   │                                                              │   │
│   │  case 'text':                                                │   │
│   │    yield { type: 'text', content: delta }                    │   │
│   │                                                              │   │
│   │  case 'tool_use':                                            │   │
│   │    ├─ StreamingToolExecutor.addTool(block, assistantMsg)  │   │
│   │    └─ yield { type: 'pending_tool_use' }  // 悬停预览     │   │
│   │                                                              │   │
│   │  case 'message_start':                                       │   │
│   │    ├─ yield { type: 'message_start', ttftMs }                │   │
│   │    └─ pushApiMetricsEntry(ttftMs)  // 追踪 TTFT           │   │
│   │                                                              │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                              │                                        │
│                              ▼                                        │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │ toolExecutor.getRemainingResults()                           │   │
│   │  ├─ processQueue()  // 扫描可执行工具                      │   │
│   │  ├─ executeTool()   // 执行单个工具                         │   │
│   │  ├─ 权限检查 → allow / deny / prompt                         │   │
│   │  ├─ 并发安全判断 → 并行 / 串行                              │   │
│   │  └─ yield { message: toolResult }  // 追加到 messages[]   │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                              │                                        │
│                              ▼                                        │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │ 决策点: 是否继续循环?                                        │   │
│   │                                                              │   │
│   │ 条件 1: 有新工具调用? (StreamingToolExecutor.tools 有新完成)│   │
│   │   └─ Yes → 追加 tool_result 到 messages → 回到 apiClient   │   │
│   │         createMessage() [继续循环]                           │   │
│   │                                                              │   │
│   │ 条件 2: max_output_tokens 错误?                              │   │
│   │   └─ Yes → maxOutputTokensRecoveryCount++                    │   │
│   │         若 < 3 → ESCALATED_MAX_TOKENS → 回到 apiClient    │   │
│   │         否则 → 返回 error_max_turns                          │   │
│   │                                                              │   │
│   │ 条件 3: tokenBudget 超限?                                    │   │
│   │   └─ Yes → budgetTracker.check() → continue / stop           │   │
│   │                                                              │   │
│   │ 条件 4: 达到 maxTurns?                                       │   │
│   │   └─ Yes → 返回 error_max_turns                              │   │
```

```
│   │                                                          │   │
│   │ 条件 5: API 错误（可重试）？                              │   │
│   │   └─ Yes → yield SDKStatus('retrying') → 重试             │   │
│   │                                                          │   │
│   │ 条件 6: 无新工具调用（API 返回最终文本响应）                │   │
│   │   └─ Yes → 追加 assistantMessage → 返回 Terminal{done}    │   │
│   └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
                          Terminal { done }
                                    │
                                    ▼
                      notifyCommandLifecycle('completed')
                      return terminal
```

## 20.2 工具并行/串行决策流程

```
API 返回 N 个 ToolCall
        │
        ▼
┌──────────────────────────────────────┐
│ findToolByName(tools, block.name)    │
│  ├─ 工具存在 → 检查 inputSchema      │
│  └─ 工具不存在 → 返回 "No such tool"错误 │
└──────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────┐
│  isConcurrencySafe 评估              │
│   ├─ parsed = tool.inputSchema.safeParse(input)│
│   ├─ 若解析失败 → isConcurrencySafe = false  │
│   └─ 若解析成功 → tool.isConcurrencySafe(parsed.data)│
│       └─ 抛异常 → isConcurrencySafe = false   │
└──────────────────────────────────────┘
                    │
                    ▼
        ┌──────────┴──────────┐
        │                     │
        │  isConcurrencySafe  │  !isConcurrencySafe
        │       = true        │      = true
        │                     │
        ▼                     ▼
┌────────────────┐     ┌────────────────────┐
│ 进入 Concurrent│     │ 进入 Non-Concurrent│
│  执行组        │     │   独占执行组        │
└────────────────┘     └────────────────────┘
        │                     │
        ▼                     ▼
┌────────────────┐     ┌────────────────────┐
│ 检查执行槽位   │     │  等待当前执行组完成  │
│ 有空位?        │     │  → processQueue()  │
│  ├─ Yes →      │     │    break 扫描循环  │
```

```
│   │    执行工具   │      └──────────────────┘
│   └─ No →       │                │
│     跳过本次    │                ▼                │
│   (后续有空位) │      ┌──────────────────┐      │
└──────────┘      │ executeTool()    │      │
          │       │ (立即执行)        │      │
          │       └──────────────────┘      │
          │                │                │
          └────────────────┴────────────────┘
                           │
                           ▼
                  ┌──────────────────────┐
                  │ tool.status = 'completed' │
                  │ results → 追加到 messages │
                  │ processQueue() 继续扫描  │
                  └──────────────────────┘
```

## 20.3 多 Agent 协调流程

```
Coordinator 收到任务 T
        │
        ▼
Task Decomposition (T → {S1, S2, S3, ..., Sn})
        │
        ▼
┌──────────────────────────────────────────┐
│ 依赖分析 → 构建并行组                      │
│                                          │
│  Group 1: [S1, S2]  // 无依赖，并行       │
│  Group 2: [S3]      // 依赖 Group 1       │
│  Group 3: [S4, S5]  // 依赖 Group 2       │
└──────────────────────┬────────────────────┘
                       │
        ┌──────────────┴──────────────┐
        ▼                             ▼
  ┌──────────────┐          ┌──────────────┐
  │ spawn Agent() │          │ spawn Agent()│
  │ for S1        │          │ for S2        │
  │ subagent_type │          │ subagent_type│
  │ = "worker"    │          │ = "worker"    │
  └──────┬───────┘          └──────┬───────┘
         │                         │
         ▼                         ▼
  ┌──────────────────┐    ┌──────────────────┐
  │ Worker 1         │    │ Worker 2         │
  │ query() loop     │    │ query() loop     │
  │ tools = Worker   │    │ tools = Worker   │
  │   Allowed Tools  │    │   Allowed Tools  │
  └──────────┬───────┘    └──────────┬───────┘
             │                        │
             └────────────────────────┘
                          │
                          ▼
```

```
<task-notification> XML
            │
            ▼
TaskOutput 收集 → aggregateResult()
            │
            ▼
Coordinator 分析结果
            │
            │
┌──────────────────────────┐
│  组 1 完成?              │
│  ├─ Yes → 启动 Group 2 │
│  └─ No → 等待           │
└──────────────────────────┘
            │
            ▼
重复直到所有组完成
            │
            ▼
synthesizeResults(allResults)
            │
            ▼
返回最终报告给用户
```

---

# 21. 总结

## 21.1 Agent 算法核心要点

1. **AsyncGenerator 模式**：整个系统的流式处理基于 TypeScript Generator + `yield*` 委托。这使得 `query()` 可以透明地流式转发 API 事件，同时在需要时注入中间逻辑（如权限检查、压缩）。
2. **两层工具编排**：
   - **API 端**（API 一次返回多个 ToolCall）：由 `StreamingToolExecutor` 决定并行/串行
   - **执行端**（工具可能继续触发更多 ToolCall）：形成递归循环直到 API 不再请求工具
3. **Token 预算作为一等公民**：从 `getAutoCompactThreshold`（精确到个位数的阈值计算）到 `MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3` 的熔断设计，整个系统围绕 Token 预算运转。
4. **分层 Agent 架构**：
   - **Main Agent**：用户主对话，拥有全部工具
   - **LocalAgent**：子进程隔离，文件系统级安全
   - **RemoteAgent**：跨 CCR 远程边界
   - **InProcessTeammate**：零开销内存级 Agent
   - **Coordinator**：管理多个 Worker 的元 Agent
5. **压缩即服务**：上下文压缩不仅是内存优化，更是一种精确的 Token 经济学实现——保留最近 5 个文件（5000 tokens/个）、5 个 Skill（5000 tokens/个），其余历史以摘要替代。
6. **记忆作为行为指导**：记忆系统（user/feedback/project/reference 四类）不是简单存储，而是要求包含 `**Why:**` 和 `**How to apply:**` 字段，使记忆具备可执行的上下文。

## 21.2 关键文件索引

| 文件 | 行数 | 核心职责 |
| --- | --- | --- |
| `query.ts` | ~800+ | `query()` 异步生成器 + `queryLoop()` 状态机 |
| `QueryEngine.ts` | ~1300+ | `QueryEngine` 类，API 封装，对话管理 |
| `tools/AgentTool/AgentTool.tsx` | ~1400+ | 子 Agent 启动，验证，异步注册 |
| `tools/AgentTool/runAgent.ts` | ~1000+ | 本地 Agent 执行引擎 |
| `services/tools/StreamingToolExecutor.ts` | ~500+ | 工具并行/串行编排引擎 |
| `coordinator/coordinatorMode.ts` | ~400+ | Coordinator 系统提示词 |
| `services/compact/autoCompact.ts` | ~350+ | Auto Compact 算法，精确阈值 |
| `services/compact/compact.ts` | ~350+ | 压缩执行核心算法 |
| `tasks/LocalAgentTask/LocalAgentTask.tsx` | ~600+ | 异步 Agent 生命周期管理 |
| `state/AppStateStore.ts` | ~500+ | 全局 React 状态管理 |
| `memdir/memdir.ts` | ~300+ | 记忆系统（存取 + 格式） |
| `tools/AgentTool/builtInAgents.ts` | ~70 | 内置 Agent 动态注册 |
| `tools/AgentTool/built-in/generalPurposeAgent.ts` | ~35 | 默认 Agent 定义 |
| `query/tokenBudget.ts` | ~100+ | Token 预算追踪 |
| `constants/prompts.ts` | ~200+ | 系统提示词常量 |
| `tools/AgentTool/constants.ts` | ~15 | Agent 常量定义 |

## 21.3 特色工程设计

| 设计模式 | 应用场景 | 实现位置 |
| --- | --- | --- |
| **Generator 异步流** | API streaming / 工具结果流 | `query.ts` |
| **Dependency Injection** | 依赖注入（可测试性） | `query/deps.ts` |
| **Feature Flag** | A/B 测试，灰度发布 | Statsig GrowthBook |
| **Circuit Breaker** | 压缩失败熔断 | `autoCompact.ts` |
| **Child AbortController** | 层级取消（父子 Agent） | `runAgent.ts` |
| **AsyncGenerator 委托** | 流式事件透传 | `yield* queryLoop()` |
| **Memory File Index** | 记忆压缩索引 | `memdir/MEMORY.md` |
| **Tool Registry** | 工具全局注册和查询 | `Tool.ts` |
| **Zod Schema 验证** | 工具输入参数验证 | 各 Tool 定义 |